LinTEX: Enhancing LATEX document accessibility for screen reader users

by

Mohammad Danyal

In partial fulfilment of the
requirements for the degree of
Bachelor of Science
in
CS408: Individual Project

Department of
Computer and Information Sciences

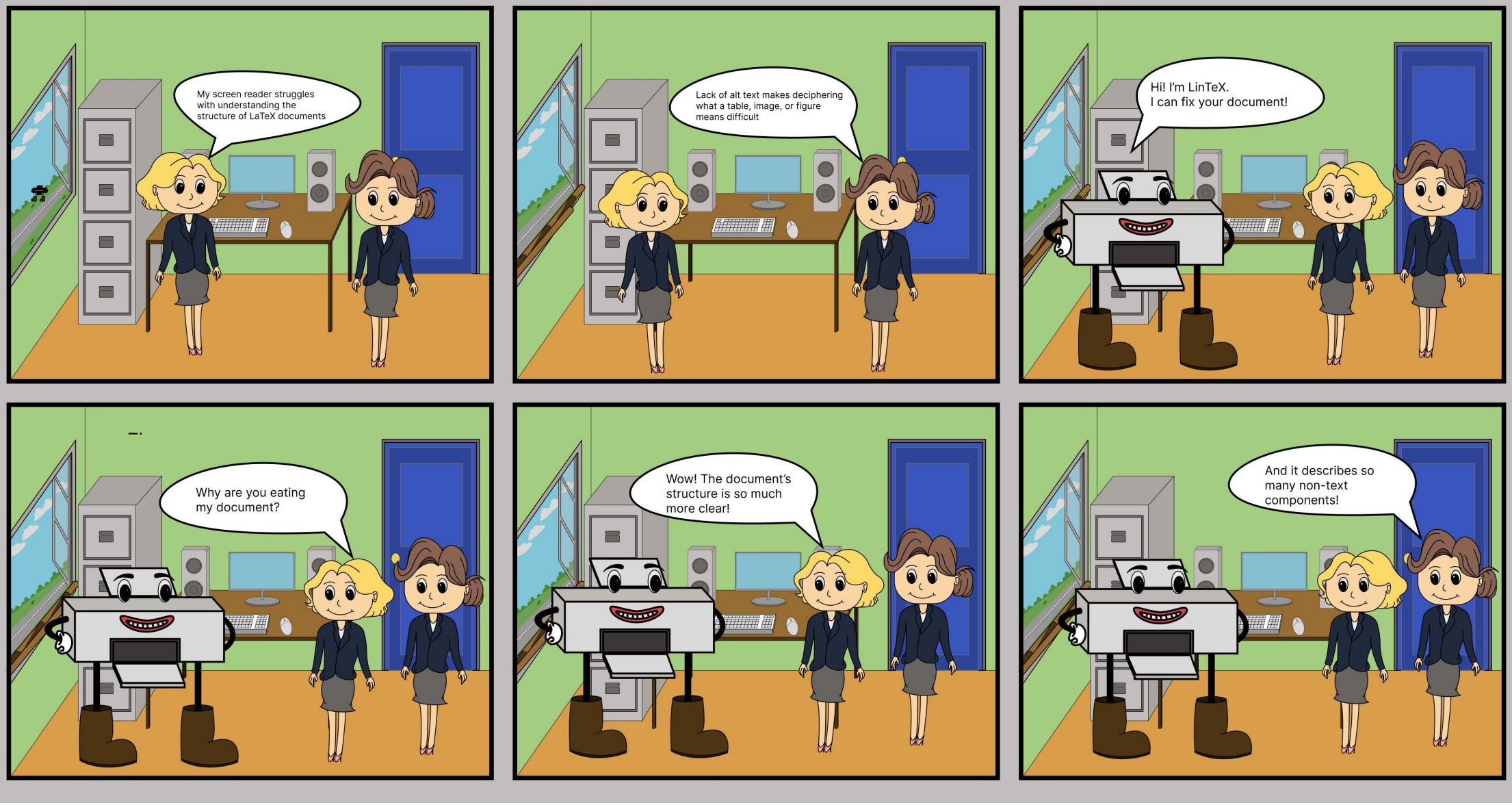
April, 2026

Abstract

Despite attempts to improve the screen reader accessibility of LaTeX through LuaLaTeX, frequent use of the outdated PDFLaTeX leads to many invalid workarounds for accessibility and a web of misinformation found online. With considerations towards appropriate resources, this project identified suitable areas in which screen reader-focused accessibility could be improved in a LaTeX document.

LinTeX achieves this by identifying lacking aspects of screen reader-focused accessibility features in LaTeX documents and suggesting solutions, often in the form of tagging and image descriptor implementations, that minimise the impact on the end user.

A user study was conducted that identified overall success with LinTeX meeting its targets of improving the accessibility of a LaTeX document for screen reader users and being a generally accessible and maintainable tool.


My screen reader struggles with understanding the structure of LaTeX documents
Lack of alt text makes deciphering what a table, image, or figure means difficult
Hi! I'm LinTeX.
I can fix your document!
—.
Why are you eating my document?
Wow! The document's structure is so much more clear!
And it describes so many non-text components!

Acknowledgements

I would like to thank my supervisor, Brian Mitchell, for his continuous guidance and support throughout the project. His depth of knowledge in related topics and the management of the project overall had vastly improved the project outcome. Although Brian would likely say luck is just a probability taken personally, I consider myself lucky to have come across and taken an interest in his project description.

I would like to provide further thanks to my friends and family who helped provide advice and support throughout the critical periods of the project. Their support was invaluable.

Last but not least, I would like to give thanks to the anonymous participants who went through the trouble of attending my user study, despite the hassles of it being an in-person session.

Contents

Overview Diagram i

Acknowledgments ii

Glossary v

1 Introduction 1
- 1.1 Identifying LaTeX's key screen reader-focused accessibility problems 1
- 1.2 Why accessibility is important 1
- 1.3 Why LuaLaTeX provides better accessibility than PDFLaTeX 2
- 1.4 Aims and objectives 2
- 1.5 Research questions 3

2 Background Literature 4
- 2.1 Choice of programming language 4
- 2.2 Choosing a PEG library under the Go programming language 5
- 2.3 Choice of frameworks and libraries 5
- 2.4 Guidelines for accessibility 8
- 2.5 Choosing a software development methodology 8
- 2.6 Screen reader choice 9
- 2.7 User study design choices 10
- 2.8 GUI design considerations 11

3 Specification & Design 12
- 3.1 Agile sprints 12
- 3.2 Key engineering requirements 12
- 3.3 LinTeX supported accessibility 12
- 3.4 Design patterns used 13
- 3.5 Error handling 14
- 3.6 Documentation 15

4 Implementation 16
- 4.1 Technologies used 16
- 4.2 PEG parsing 16
- 4.3 Verification & validation 18

5 User study 19
- 5.1 Study design 19
- 5.2 Hypotheses 20
- 5.3 Results 20
- 5.4 Analysis 23
- 5.5 Hypotheses testing and discussion 27

6 Discussion 28
- 6.1 Objectives discussion 28
- 6.2 Research question discussion 28

7 Conclusions **29**
7.1 Project summary . . . 29
7.2 Project limitations . . . 29
7.3 Achievements . . . 30
7.4 Future work . . . 30
7.5 Reflection . . . 31
7.6 Final thoughts . . . 31

A Appendix **35**
A.1 Ethical approval . . . 35
A.2 Ethics application . . . 38
A.3 Participant information sheet . . . 41
A.4 Consent form . . . 45
A.5 User study script . . . 47
A.6 Inaccessible document . . . 51
A.7 Accessible document . . . 54
A.8 PEG script . . . 57
A.9 Agile sprint focuses . . . 65
A.10 Functional requirements . . . 66
A.11 Non-functional requirements . . . 67
A.12 Error list . . . 68

List of Figures

1 Building an accessible document . . . 14
2 Error handling in LinTEX . . . 14
3 LaTeX document structure . . . 17
4 LaTeX line construct structure . . . 18
5 Comparing screen reader output quality ratings before and after applying LinTEX . . . 21
6 Paired t-test formula . . . 22
7 Participants likelihood to support screen reader users in the future . . . 24

List of Tables

2 Languages and related PEG frameworks . . . 4
3 Languages and related website development support . . . 5
4 Comparison between PEG and Pigeon . . . 6
5 Screen reader pricing status and OSes . . . 9
6 Memorable aspects of document extracts . . . 20
7 Difficult aspects from the initial and final listenings . . . 22
8 Simple aspects from the initial and final listenings . . . 23
9 LinTEX usability . . . 24

Glossary

accessible document	an accessible LaTeX document created by applying LinTeX to the inaccessible document. This document can be found in section A.7. 8, 19, 20, 25, 27
actual text	is used to repeat text extracts contained within a graphic. v, 8, 13
AI	artificial intelligence 1
alternative text	descriptive text used in place of non-text components for accessibility purposes. v, 8, 13, 18, 29, 65
artifact	an indicator to a screen reader to skip an image in its reading 8, 13
AST	abstract syntax tree 6
BDD	behaviour-driven development 8, 9
CDN	content delivery network 7
CGI	Common Gateway Interface 4
CLI	command-line interface 7
CMD	command prompt for Windows 7, 16
CSS	Cascading Style Sheet 7, 16
GUI	graphical user interface 1, 65
HTML	Hypertext Markup Language 6, 7, 16
HTTP	HyperText Transfer Protocol 7
IDE	integrated development environment 11, 30
image descriptor	an umbrella term to reference both alternative text and actual text. i, 1, 2, 8, 12, 30
inaccessible document	a LaTeX document that has intentionally been made inaccessible for screen reader users, which can be found in section A.6. v, 11, 19, 20, 25, 27
JS	JavaScript 7
likert scale datum	a scale, usually with 5 or 7 points, indicating a participant's agreement or disagreement with some topic (Sullivan 2013). 10, 11
metadata	data about data, such as a file's structure. 22, 25
MVP	minimum viable product 65
npm	Node package manager 7
OS	operating system iv, 9

PDF	Portable Document Format 1, 2, 6, 12, 13, 30, 67
PEG	parsing expression grammar iv, 4–6, 8, 15–17
PIS	participant information sheet 65
qualitative data	data that cannot be represented numerically, such as descriptions, statements or quotes (National Library of Medicine 2026a). 10, 11
quantitative data	data that can be represented in numerical form (National Library of Medicine 2026b). 11
right-tailed test	a test for whether the hypothesis should be rejected (Exeter 2025). 27
tagging	the process of providing data about the structure of a document to be used in its audio output. i, 1, 2, 12, 29, 30
TDD	test-driven development 8, 9
think-aloud session	users think aloud while performing a given task, with the goal of identifying areas of improvement in the usability of the given context. 10, 11
UI/UX	user interface and user experience 11
WSL	Window's Subsystem for Linux 6

1 Introduction

1.1 Identifying LaTeX's key screen reader-focused accessibility problems

Similar to a tool like Microsoft Word, LaTeX is a tool used to process documents. LaTeX does this in a programmatic manner, likening it to a programming language, whereas Microsoft Word has a graphical user interface (GUI) which simplifies the process for an ordinary user. However, LaTeX prevails in its ability to process scientific documents, which may require graphs, images, tables, or advanced mathematical capabilities.

LaTeX struggles to implement accessibility for screen readers, largely due to it not originally being built for that purpose, as explained in section 1.3. Additionally, LaTeX is broken by design, originally created as a set of macros built on the TeX typesetting engine that evolved into the LuaLaTeX that we work with in modern times.

Additionally, Portable Document Format (PDF) files are built not to be changed, which adds difficulties with attempting to add tagging or image descriptors.

1.2 Why accessibility is important

Accessibility has the potential to assist all users, even if it is aimed at a few focus groups. One such case is where an environment, such as a noisy train, restricts the user from hearing the audio output of a video. In this context, an accessibility feature such as subtitles on a video, ideally without artificial intelligence (AI) generation, can greatly enhance a user's experience.

For example, in Northern Ireland alone, there are an estimated 8,000 users with sight loss (University 2026). These users may require the assistance of screen readers or someone else to read content out to them, which can add to their stress, with the digital context simply not being built with their needs in mind.

Users with disabilities are over 50% more likely to face difficulties due to a lack of digital inclusion (University 2026), which highlights the severity of its impact on a website's overall traffic. These users should not be an afterthought of the website's design, but rather a key focus.

1.3 Why LuaLaTeX provides better accessibility than PDFLaTeX

PDFLaTeX was created to be a media printer, which produced PDF documents with the sole purpose of being printed out. Due to its innate design, this means it lacked support for screen reader users. PDFLaTeX also frequently utilises outdated, or frankly broken, workarounds to implement accessibility, such as the accessibility package (Clifton 2019), which has been deprecated since February 2021. Additionally, as its focus has shifted to maintenance rather than development, PDFLaTeX is unsuitable for accessibility where requirements change frequently, making it an outdated TeX engine, which unfortunately still sees frequent usage.

The introduction of XeTeX improved font handling in LaTeX, yet its lifetime was short-lived, with it almost being considered an evolutionary dead end, with the advent of LuaLaTeX.

Unlike PDFLaTeX , LuaLaTeX was created for the purpose of creating electronic documents, which provides its development a suitable reason to support screen reader users through the use of image descriptor and tagging. Our tool, LinTeX, utilises LuaLaTeX's accessibility support, with emphasis placed on screen reader users.

1.4 Aims and objectives

This project aimed to enhance LaTeX document accessibility, with a focus placed on screen reader users. This was achieved by creating a prototype, LinTeX, which analyses a LaTeX document to identify where to apply image descriptors and tagging.

The following objectives were created for this context:

OBJ1 The tool, LinTeX, should be maintainable, as accessibility often requires changes to its management.

OBJ2 LinTeX should improve a document's audio output in terms of its contents, when read by a screen reader.

OBJ3 LinTeX should provide useful and descriptive error messages.

OBJ4 LinTeX should be reasonably accessible, following the accessibility guidelines defined in section 2.4.

1.5 Research questions

The following research questions cover the research-focused aspects of this project:

RQ1 What considerations are necessary to make a LaTeX document accessible for screen reader users?

RQ2 Why does LaTeX suffer a lack of accessibility for screen reader users?

2 Background Literature

2.1 Choice of programming language

There were many considerations when deciding what programming language to write LinTEX in. The languages chosen were based on those I had prior experience in, to ensure minimal hindrance from the language itself to the project development. An important factor is ensuring LinTEX's maintainability in the future. Therefore, only languages that have actively maintained parsing expression grammar (PEG) libraries will be considered, as can be viewed in Table 2. The most recent update times provided in Table 2 are in reference to 24th November 2025.

Table 2: Languages and related PEG frameworks

Language	PEG library link	PEG library is in development
Python	https://pythonhosted.org/pyPEG2/	No, last updated in October 2015
Python	https://github.com/erikrose/parsimonious	Yes, last updated in November 2025
JavaScript	https://github.com/pegjs/pegjs	No, last updated in November 2019
JavaScript	https://peggyjs.org/	Yes, last updated in September 2025
Go	https://github.com/pointlander/peg	Yes, last updated in November 2025
Go	https://github.com/mna/pigeon	Yes, last updated in November 2025
C	N/A	N/A
Java	https://github.com/sirthias/parboiled	Yes, last updated in November 2025
C#	https://code.google.com/archive/p/peg-sharp/	No, last updated in May 2011
C#	https://github.com/otac0n/Pegasus	No, last updated in December 2023
C++	https://github.com/yhirose/cpp-peglib	Yes, last updated in July 2025
PHP	https://github.com/hafriedlander/php-peg	No, last updated in May 2014
TypeScript	https://github.com/metadevpro/ts-pegjs	No, last updated in November 2024
TypeScript	https://github.com/EoinDavey/tsPEG	Yes, last updated in November 2025
Dart	https://github.com/darmie/dart-peg	No, last updated in July 2015

Based on the current criteria, we can rule out C, C#, PHP, and Dart.

As many languages remain, there is a need for stricter criteria: suitability to be used as a web application and compatibility with the University of Strathclyde's development web server (devweb) system.

It should be noted that while Go, C, C#, and C++ are not directly compatible with devweb, as referenced in Table 3, they can be supported through the use of Common Gateway Interface (CGI) scripts, and consequently should not be fully disregarded for this alone. However, based on their support for web development, C, Java, C#, and C++ will have a lower priority for being a chosen language.

Table 3: Languages and related website development support

Language	Web development support	DevWeb support
Python	Yes	Yes
JavaScript	Yes	Yes
Go	Yes	No
C	No, except for rare exceptions	No
Java	Yes, but not generally used	Yes
C#	Yes, but not generally used	No
C++	Yes, but not generally used	No
PHP	Yes	Yes
TypeScript	Yes	Yes
Dart	Yes	Yes

This leaves a focus on JavaScript, Go, and TypeScript. However, JavaScript will be removed from consideration as a strictly typed language is preferred for our context. Therefore, leaving Go and TypeScript as the main considerations for this project. Lastly, as I have more experience and a stronger preference towards Go as opposed to TypeScript, it was the chosen programming language for the project.

2.2 Choosing a PEG library under the Go programming language

Go has two libraries that are used in relation to PEG systems: Pigeon by Martin Angers (mna) and PEG by Andrew Snodgrass (pointlander). Direct comparisons with core components of both have been made in Table 4.

While examples are relatively similar, the simplicity of installation for Pigeon as opposed to PEG, combined with the depth of documentation both in generated code and outwith it, makes Pigeon a more ideal option. Additionally, its greater number of features gives LinTEX more to make use of. Therefore, Pigeon was the chosen library for the project.

2.3 Choice of frameworks and libraries

2.3.1 Backend frameworks and libraries

There were three options for web frameworks and packages, namely Gin (Martínez-Almeida 2014), Gorilla (The Gorilla Authors 2023), and net/http (The Go Authors 2009c). For LinTEX, net/http was chosen for the key reason that it had the least barriers to entry, as there was a provided guide on the Go development website that was utilised in the

Table 4: Comparison between PEG and Pigeon

	Pigeon	PEG
Installation	Requires running a `go install` command and setting up environment variables.	Requires supporting bash scripting (through Window's Subsystem for Linux (WSL) for Windows users), as well as running a `go install` command, cloning the repository, and running `go generate && go build`.
Documentation	Contains a Godoc page, wiki page, an examples subdirectory, and a detailed README.md providing what PEG features are supported, command line interface usage, installation, and examples.	Contains a detailed README.md and a PEG file syntax file containing what PEG features are supported, installation steps, and example projects links.
Generated Code	Generated Go code can be used to configure settings on the parser, view statistics or error information, debug, enable or disable memoization, and parse readers, files, and strings. Additionally, the generated code is well documented, with several comments surrounding usage of public functions.	Generated Go code can be used to run the PEG script on a provided string, debug and output the abstract syntax tree (AST), output errors in parsing, and enable or disable memoization.
Examples	Provides three direct examples: detecting indentation in files, parsing JSON files, and a calculator example. Examples were kept short (regarding PEG scripts) with the calculator and indentation examples showcasing usage via a main function.	Provides two project links for examples on PEG usage: a natural date/time parser and a scientific names parser. Both projects have large PEG scripts, with most lines being simple to read. The natural date/time project was intuitive to use and understand program flow.

project setup. However, in hindsight, more consideration towards what web framework to use would have had a great impact on the project as a whole, as net/http is a lower-level web framework than Gin or Gorilla, consequently requiring more management on LinTEX's end. Some examples of difficulties that occurred are a lack of POST requests being easily supported and necessitating that server files are explicitly stated as opposed to implicitly defined within a given context.

The html/template (The Go Authors 2009b) package was included in the project. This package provided the ability to load Hypertext Markup Language (HTML) files through Go, while allowing information to be provided to them. The alternative choice was to provide a raw print output to the client, which made the package a far clearer winner.

For the project to be able to compile LaTeX documents to convert them into PDFs, it was necessary to either use a pre-built package or to compile LaTeX documents locally. Pre-built packages that were found include go-latex (T. l. Authors 2020) and latex-fast-compile (Kpym 2020). It was necessary to rule out go-latex as it emulated a LaTeX compiler, rather than directly linking to one. Additionally, it lacked documentation and necessary features,

making it unsuitable. Latex-fast-compile was also unsuitable due to using PDFLaTeX rather than LuaLaTeX , which was desired.

Consequently, it became necessary to compile LaTeX documents directly. This was done through the use of command prompt for Windows (CMD), which allowed the 'lualatex' command to be run, therefore compiling the requested file. It did, however, require that the hosting server has a TeX typesetting engine in their backend, for the LuaLaTeX engine. The os/exec (The Go Authors 2009a) package allowed for a CMD shell to be created within Go code, consequently allowing the 'lualatex' command to be run. Hence, compiling the LaTeX document that the user provides.

2.3.2 Frontend frameworks and libraries

As plain Cascading Style Sheet (CSS) is difficult to work with and maintain, it was necessary to decide on a CSS framework for the project. My prior experience with Bootstrap (T. B. Authors 2011) made it a close choice, but the opportunity to learn to utilise Tailwind CSS (Tailwind Labs 2017) was more enticing and eventually won the decision.

To implement Tailwind CSS, it was necessary to install its command-line interface (CLI) through Node package manager (npm). This permitted usage of Tailwind CSS without necessitating JavaScript (JS) within the codebase, and avoided workarounds such as a content delivery network (CDN). Additionally, due to difficulties in using Tailwind CSS, it was necessary to utilise a component library built on top of it. Flowbite (Inc. 2023) was chosen as it could be used as almost plug-and-play and contained a vast amount of relevant examples.

To ensure appropriate responsiveness of LinTeX, it is necessary to be able to introduce updates in real time. For this, HTMX (Software 2020) was planned to be utilised, as it is a tool that permits HTML tags other than the 'a' and 'form' tags to send HyperText Transfer Protocol (HTTP) requests to the server. For example, it is possible to use HTMX to create a HTTP request when a given key is pressed through its event management system.

2.4 Guidelines for accessibility

The following key points have been used in the design of LinTEX to ensure its accessibility, largely following WCAG 2.2 (W3C 2018) standards.

- The website should have all non-text content contain image descriptors (WCAG 2.2: Guideline 1.1).
- The website may be operable through the keyboard alone (WCAG 2.2: Guideline 2.1).
- LinTEX should provide simple error messages.
- The website should allow for content to be distinguishable, including supporting resizing content and minimal colour usage (WCAG 2.2: Guideline 1.4).
- The website should be presentable in a simpler format if required (WCAG 2.2: Guideline 1.3).
- The website should include navigation features (WCAG 2.2: Guideline 2.4).
- The website should be predictable (WCAG 2.2: Guideline 3.2).
- The produced document by LinTEX should contain appropriate tagging.
- The produced accessible document by LinTEX should contain appropriate alternative text, actual text, or artifact.

2.5 Choosing a software development methodology

The available options under consideration for the project are the waterfall methodology, agile development methodology, test-driven development (TDD), and behaviour-driven development (BDD).

The waterfall methodology requires significant research early in the project to have success. However, as I lacked experience in PEG script usage and many related topics for this project, it was likely that changes would be needed dynamically.

TDD was the next consideration, with a personal strong bias towards it due to its usefulness in being self-documenting and saving the hassle of writing the unit tests after creating the software, which can be a more draining process. However, as Pigeon (the PEG library

chosen in section 2.2) generates Go files without a way to test individual components, it was not possible to create unit tests without it requiring an entire LaTeX document for each unit test to be made. Additionally, creating unit tests can be time-consuming, which is unsuitable for the short timeframe of this project.

BDD was removed from consideration for similar reasons as TDD, with the addition of it being more suitable for larger contexts such as game development, where it is less practical to test every feature individually.

The final methodology, agile development methodology, allowed for code to be created quickly through its sprints. Additionally, as there is no prerequisite to create unit tests before creating the features, it saved a lot of time in development for the project. Consequently, agile development methodology was chosen for this project.

2.6 Screen reader choice

Through a small number of criteria, as showcased in Table 5, it was possible to greatly reduce the choices for what screen reader to use in our context.

Table 5: Screen reader pricing status and OSes

Screen reader	Paid service	operating system support
JAWS	Yes	Microsoft Windows 10, 11, and server editions
Apple VoiceOver	No	Apple devices including iPhones, iPads, Macs, and AppleWatches
Windows Narrator	No	Windows 10 and 11
TalkBack	No	Android 6.0 Marshmallow onwards
ChromeVox	No	Primarily ChromeOS, with older versions supporting devices using Chrome browser
Supernova	Yes	Microsoft Windows 10, 11, server editions, and virtual environments
NVDA	No	Microsoft Windows 8.1 to 11

As a student, I do not have the leeway to purchase subscriptions for several screen readers, ruling out those that require payments, even if it would just be in the form of a free trial. Additionally, there is an ethical dilemma in requiring payments for a necessary accessibility tool, which pushed me away from choosing them. Another criterion was that the Windows operating system (OS) should be supported, with emphasis placed on Windows 10 and Windows 11. These were chosen as they are the most common PC OS used, consequently targeting a larger base of users who may require screen readers.

Considering these criteria, NVDA and Microsoft Narrator were the final choices. When testing both on a placeholder document, emulating the experience of a screen reader user

working with LaTeX, I found that there was more difficulty with Windows Narrator in terms of having it read the correct extract on the document, often requiring using a mouse to get to the correct location. Inversely, NVDA read the correct extract in full detail. Hence, by process of elimination to a degree, NVDA was the chosen screen reader for LinTeX.

Adobe Acrobat Reader was used with NVDA to make the documents easier to navigate in more complex contexts, such as those where the document may not be read as simply as top-to-bottom, left-to-right.

2.7 User study design choices

2.7.1 User study structure design

A combination of a think-aloud session and questionnaires was utilised in the creation of the user study.

A think-aloud session was performed where users think aloud while performing a given task, with the goal of identifying areas of improvement in the usability of the given context. This was utilised to acquire qualitative data regarding the usability of LinTeX, with focus placed on most affected areas in Table 9.

It was deemed necessary to compare a LaTeX document before and after applying LinTeX's changes to determine whether LinTeX had a measurable impact on the screen reader-related accessibility of the documents. This comparison was created through a listening of the document's screen reader output at a given stage and a questioning stage immediately after the relevant listening. This utilised the screen reader selected in section 2.6

Additionally, it was possible that the participant's experience level in related topics would have an impact on their answers to questions or experience during the think-aloud session, consequently necessitating a small questionnaire to gauge experience levels.

2.7.2 User study question design

Factors that may influence a user's experience in the user study include their experience level in LaTeX, using screen readers, and listening to audio recordings of speaking. Likert scale data were used to gauge these experience levels in a numerical method, providing

quantitative data.

Qualitative data were obtained from open-ended questions, with a clear focus placed on each question, to minimise confusion.

To gauge an understanding of how truly impactful an inaccessible document's impact was on a screen reader user's experience, participants were questioned on what they could recall from the listening. However, as the participant will have worked with the document directly during the think-aloud session, it is redundant to include it in the final questioning.

It is necessary to understand what components of the relevant listening were found to be simple or difficult. However, these should be asked separately to ensure no ambiguity about what is being asked. The participant will be asked to provide their opinion on how well the screen reader audio output was understood in terms of its contents, using a likert scale datum to provide quantitative data that can be analysed.

Participants were asked if they were likely to accommodate screen reader users in the future, to gain an understanding of whether LinTEX had made doing so appear to be an easier process.

2.8 GUI design considerations

LinTEX took strong inspiration from integrated development environments (IDEs) with regards to its user interface and user experience (UI/UX) design. This is because LATEX in structure is similar to a programming language, even being Turing complete (Murrish 2012). Hence, an environment suitable for programming is suitable to program LATEX in, with some special considerations put aside for its quirks. For example, depending on the user's screen size and preferences, they may wish to have the output of the LATEX documents taking up half of the screen, a separate browser window or monitor, or out of the way until compilation.

Additionally, online code editors such as LeetCode (Tang 2015), HackerRank (Ravisankar **and** Karunanidhi 2011), and CodeWars (Doctor **and** Hoffner 2012) were similar in nature to what LinTEX would like to achieve regarding its UI/UX, and were consequently used as inspiration.

3 Specification & Design

3.1 Agile sprints

An agile sprint is a short timeframe where dedicated efforts are focused on a small number of goals. They were utilised for several components of the project, with not only a usage in the software engineering side, but also on the project management end. A list of these sprints can be found in the appendix, namely section A.9.

3.2 Key engineering requirements

3.2.1 Key functional requirements

The key functional requirements include:

- LinTEX must allow users to upload one LATEX file, preview the generated PDF file, and download it if they request.
- LinTEX must detect and recommend fixes for tagging and lack of image descriptor.
- LinTEX must provide useful error messages, in accordance with objective **OBJ3**.

The full list of functional requirements can be found in the appendix, namely section A.10.

3.2.2 Key non-functional requirements

Non-functional requirements include:

- The website should be reasonably efficient.
- The website should be flexible with supported browsers and desktop screen sizes.
- The website should be reasonably accessible.
- LinTEX should have thorough documentation describing its usage and code. This conforms to the objective **OBJ1**

The full list of functional requirements can be found in the appendix, namely section A.11

3.3 LinTEX supported accessibility

The key LATEX accessibility features supported were:

Document tagging The document would have tagging applied through its `\DocumentMetadata` command. This command accepts a language ('lang') key to specify the document language; tagging key to enable tagging; tagging setup ('tagging-setup') key to select the type of tagging applied to the LaTeX document; PDF standard ('pdfstandard') key to specify the PDF standard the document should follow; and the PDF version ('pdfversion') key to specify the version of the PDF. The PDF version should be compatible with the PDF standard. It should be noted that the PDF standard PDF/UA-2 has inconsistencies in version 2.0, which are predicted to be fixed in version 2.1.

Table tagging Each table should have its tagging standard applied, with the `\tagpdfsetup` command prefixing the tabular group construct, as defined in PEG parsing (section 4.2), with either 'table/tagging=presentation' in its required argument for tables that are read in reading order, or 'table/header-rows=N', where N is the row the headings, for tables where the heading is prefixed before stating a table cell's contents.

Alternative text `\includegraphics` may include alternative text applied through the optional key 'alt'.

Actual text `\includegraphics` may include actual text applied through the optional key 'actualtext'. Unlike alternative text, which has the purpose of describing the text, actual text is used to repeat text extracts within the image itself. For example, a tongue twister.

Artifact `\includegraphics` may include an artifact, applied through the optional key 'artifact' with no value or equal sign ('='). This is used to avoid a graphic's screen reader audio output for when a graphic is purely decorative.

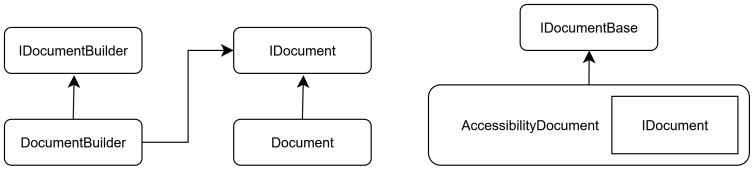
All `\includegraphics` are expected to have alternative text, actual text, or artifact.

Examples of each of these supported accessibility features can be found in section A.7.

3.4 Design patterns used

Figure 1 showcases the design patterns involved in the creation of an accessible document.

Figure 1: Building an accessible document



The Builder design pattern (Shvets 2014b) was used to create a document that conforms to the 'IDocument' interface, rather than a concrete 'Document' class. This was necessary to work together with the Adapter design pattern (Shvets 2014a) to ensure a concrete 'AccessibleDocument' class could be created, with it containing the original document conforming to the 'IDocument' interface. This promoted the reusability of the concrete 'Document' for other contexts, which would not have been possible if the builder pattern had created a concrete 'AccessibleDocument' from the start.

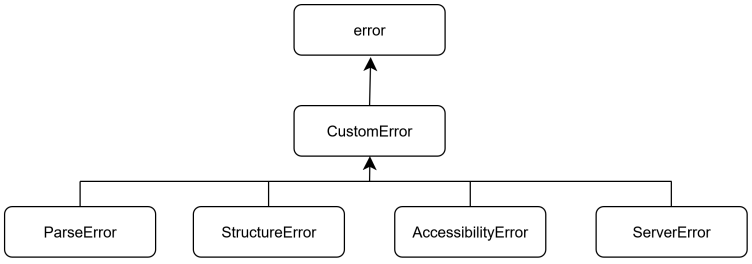
The Composite design pattern (Shvets 2014c) was utilised for the handling of the document content, as showcased in Figure 3. This ensured that both group and line constructs could be treated uniformly through a common interface.

PEG parsing (section 4.2) describes how arguments were written to conform to a common interface, ensuring compliance with the dependency inversion principle (Noback 2018).

3.5 Error handling

Below is a diagram showcasing the general structure of error management in LinTEX:

Figure 2: Error handling in LinTEX



Error handling in LinTEX follows the dependency inversion principle (Noback 2018). In Go, all errors are written to the 'error' interface. However, it was necessary to create our own interface, 'CustomError', to have special handling of data, while still conforming to the

'error' interface. From that point, it is relatively simple to have error classes, which allow for errors to be categorised, to conform to the 'CustomError' interface.

Another important point is to avoid the repetition of data in an error. For example, the 'CustomError' interface implements separate `LongError()` and `Error()` methods. Repeating the output of the `LongError()` in several locations would likely lead to mistakes and additional work in keeping them updated. Hence, a custom function type, 'CreateError', was created to compile the error while only necessitating the location information of the error. This worked by creating a helper function that returned the 'CreateError' function type, and provided the consistent data between errors to the helper function, such that the 'CreateError' function type would only require the location information of the error. The full error list can be found in the appendix in section A.12.

Lastly, it is worth noting that there are two methods of providing errors in LinTEX: providing a short, but contextualised error message providing the error extract and location information; and providing a long and descriptive error message without information regarding the location the error had occurred.

3.6 Documentation

Thorough documentation has been provided in the form of comments in code, with special emphasis on the PEG script in section A.8. Additionally, the 'README.md' contains thorough steps focused on prerequisite installations, installation steps, and steps for contributing to the project.

4 Implementation

4.1 Technologies used

Backend technologies used include Pigeon (Angers 2025) for parsing expression grammar (PEG) scripting, which generated a Go file; net/http Go package (The Go Authors 2009c) to handle client-server communication; html/template Go package (The Go Authors 2009b) to provide HTML output to the client; and os/exec (The Go Authors 2009a) to compile LaTeX via the command prompt for Windows (CMD).

Frontend technologies used include Tailwind CSS (Tailwind Labs 2017) to avoid repetition of Cascading Style Sheet (CSS) through use of components and Flowbite (Inc. 2023) to expand on its base components library.

Additionally, Go, Hypertext Markup Language (HTML), Pigeon (Angers 2025) PEG script, and CSS were the languages used in the project.

Lastly, LinTeX implements a server-client relationship. This means the client sends a request for resources and the server provides those resources if it decides to accept the request, or provides an error. The net/http Go package (The Go Authors 2009c) requires all directories and files on the server be explicitly provided, and applies validation to the server route taken, both of which acts as an additional security meaasure.

4.2 PEG parsing

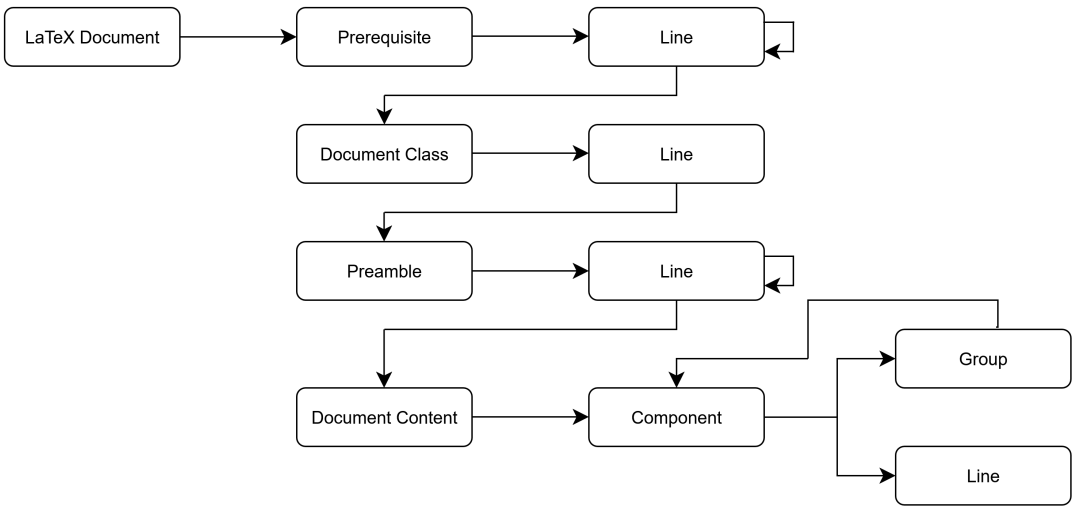
Figure 3 showcases the order in which LinTeX parses the document in its PEG script. The PEG script can be found in section A.8.

There are a few key points in reading this diagram:

- Prerequisite is the chosen name to reference content before the document class in our context.
- The line construct immediately after the document class must have the identifier 'documentclass'.
- The topmost layer for the document content must conform to a group construct with the identifier 'document'.

- Components is an interface that can conform to either group or line constructs, following the Composite design pattern (Shvets 2014c).
- A line construct references a singular LaTeX line, such as `\includegraphics[alt={Some alt text}]{image.png}`. This does not reference any LaTeX line that is involved in a grouping construct.
- A group construct references a named group starting with `\begin{<name>}` and ending with `\end{<name>}` with `<name>` swapped for the group identifier. Other methods to reference group constructs are not included in LinTeX's parsing.
- Anonymous, or unnamed, group constructs are not handled by LinTeX.
- The content between line or group constructs, that do not conform to either, is mostly ignored. For example, plain text.

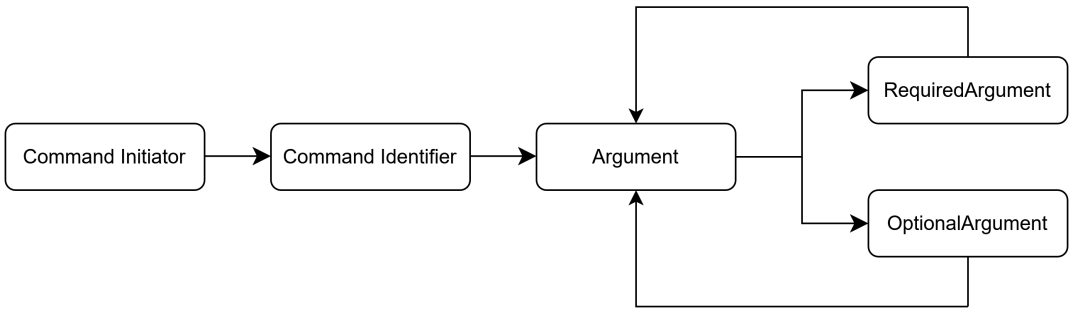
Figure 3: LaTeX document structure



In the creation of this PEG script, a top-down approach was taken. This involved looking at the general structure of a LaTeX document and breaking it down further continuously until a group construct or a line construct is reached, depending on the context.

Figure 4 showcases the general structure of a line construct. A similar structure was utilised for group constructs, with the difference that the first required argument, an argument surrounded by curly braces ('{}'), contained the group identifier. A stack was utilised in a group's depths management and error identification. A generic class, a class that can

Figure 4: LaTeX line construct structure



have its typing defined during program execution, was utilised with its typing passed during construction of the stack. This provided the advantage of making the stack code reusable for different underlying typings.

As the final parsing element, argument handling handled both optional (referenced as 'OptionArgument' in program code) and required (referenced as 'ClassArgument' in program code) arguments separately, while ensuring they conform to the same argument interface. Additionally, an argument type whose inner content aligns with a key-value pairing was created to handle some arguments in more depth. A key-value pairing references having a key and value separated by an equal sign ('=') creating a pair, and consequently having all pairs separated by a comma (',').

4.3 Verification & validation

Verification took advantage of Go's strictly typed nature, such that a custom data structure or interface was created for every reasonable context. For example, there is an interface, 'IDocument', and a 'Document' class that conforms to it, both of which are used in the handling of the overall document structure being enforced. However, minimal verification of the document content is applied at this stage, as that is outwith the scope of this project as defined in project limitations (section 7.2).

Validation is applied through the Adapter class (Shvets 2014a), 'AccessibleDocument', with its checks over the document, such as verifying the document contains tagging for its whole document and tables, as well as ensuring all images contain alternative text unless otherwise specified.

5 User study

A user study is an approach taken to understand the requirements of users in relation to your application (Center 2024). It can be used to better understand user requirements for those who will be using the system and consequently improve design decisions.

This user study was given ethical approval, as can be viewed in ethical approval (section A.1).

5.1 Study design

The user study was designed with the goal of evaluating the effectiveness of the tool, LinTEX, and identifying areas of improvement for the tool. To meet these goals, the user study was split into six key stages:

1. An introductory stage where the goal is to identify where the participant's knowledge lies in related topics.
2. An initial listening of the inaccessible document, which was produced for this user study.
3. A questioning stage based on the initial listening of the inaccessible document.
4. A think-aloud session for understanding the accessibility of LinTEX.
5. A final listening of the newly created accessible document by using LinTeX.
6. A questioning stage based on the final listening of the accessible document.

These stages allowed for a comparison of the before and after of using LinTEX, through stages 2 and 5, as well as associated questioning stages 3 and 6, allowing both qualitative and quantitative data to be obtained. Additionally, the think-aloud session provides an opportunity to judge the usability of LinTEX.

Finer details for the user study can be found in the user study script (section A.5), with the reasoning behind their choices located in the user study design choices (section 2.7).

5.2 Hypotheses

There were several hypotheses that this user study aimed to prove or disprove:

H1 The accessible document created by LinTEX showed an improvement from the inaccessible document initially used in terms of the document's accessibility for screen reader users. This hypothesis is based on the objective **OBJ2**.

H2 LinTEX is reasonably accessible. This hypothesis is based on the objective **OBJ4**.

H3 LinTEX provided useful error messages. This hypothesis is based on the objective **OBJ3**.

H4 The inaccessible document used in the initial listening is not memorable.

5.3 Results

The sample size for this user study was small, with 5 participants being recruited for it.

5.3.1 Memorable document extracts

The following table, namely Table 6, showcases the results from the question in the initial questioning (stage 3), which requests participants to summarise the document.

Table 6: Memorable aspects of document extracts

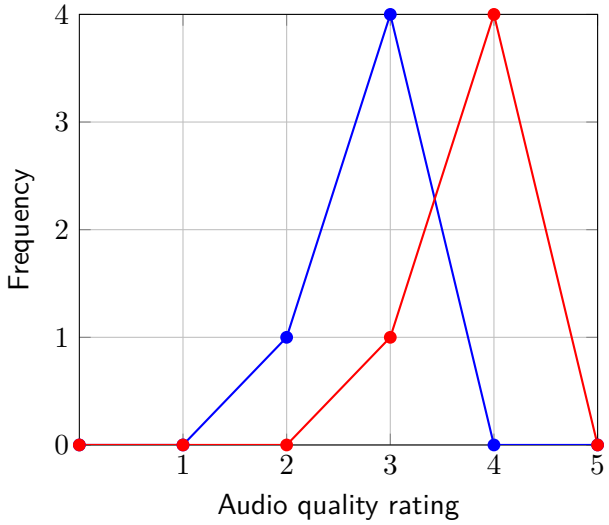
Document extract	Comment
Improving spoken skills	The majority of participants recalled spoken skills being improved through tongue twisters, yet there was minimal mentions of other techniques referenced in the inaccessible document.
House sale	A little over half of the participants had recalled a house being mentioned, with the currency and specific pricing often incorrect.
Favourite pet	All users recalled the extract about the cat being the favourite pet.
Muhammad and Noah name comparisons in Scotland	All users recalled this extract, with finer details such as: whose name was more popular, the positions of the names, and what names were in the context were often incorrect.
Grade comparisons	All users recalled this extract, with finer details such as: the friends' names, John's name, the class codes, whether the extract referenced John's friends or general students, who was assigned what grades, and the grades themselves, were often incorrect.

It should be noted that participants who had requested to pause or repeat a segment of the initial listening showcased a better recollection of the document extracts, with the average participant able to recall 4 extracts on average.

5.3.2 Accessible and inaccessible document comparisons

The following figure, namely Figure 5, showcases the comparison of Likert scale data obtained from the user study that queries the participants about the quality of the screen reader's audio output regarding its contents. This data was used to create a frequency polygon graph, for which the blue line showcases the data from the initial questioning (stage 3) and the red line showcases the data from the final questioning (stage 6).

Figure 5: Comparing screen reader output quality ratings before and after applying LinTEX



The key difficult components of both the initial listening (stage 2) and final listening (stage 5) can be found in Table 7. It should be noted that this is not an exhaustive list of all difficult components from both stages, but rather the key points that a majority of participants in the user study emphasised. For example, a few participants noted that the scale of numbers included in a table had an impact on its legibility.

The key simple components of both the initial listening (stage 2) and final listening (stage 5) can be found in Table 8. It should be noted that this is not an exhaustive list of all simple components from both stages, but rather the key points that a majority of participants in the user study emphasised. For example, many participants enjoyed the inclusion of an example tongue twister in place of the relevant image.

It should be noted that all participants required less pausing or repetition of an extract in the final listening (stage 5) compared to the initial listening (stage 2). Additionally, another

Table 7: Difficult aspects from the initial and final listenings

Initial document	Final document
Repetition of 'percent' in the second table was found to be distracting.	Repetition of 'percent' in the second table was found to be distracting.
Images were difficult to keep track of when and where they had occurred.	Tables contained too much metadata, with focus placed on 'column N' being repeated for each table cell.
Images being plain file names made it difficult to ascertain their purpose.	Document metadata could be confused with the contents of the document, with emphasis placed on column numbers being mixed with name positions in the first table.
Headings and pages did not have sufficiently long pauses before beginning.	
Tables were read too fast with no pauses between table cells.	
The table containing names is viewed from top to bottom, whereas the screen reader reads from left to right.	

point of note is that tables were a frequent aspect that required pausing or repetition.

5.3.3 Applying student's paired t-test

The student's paired t-test was developed by William Sealy Gosset in 1908 (Wadhwa **and** Marappa-Ganeshan 2023). Its purpose, in our context, is to determine if the difference between our audio output quality Likert scale data in Figure 5 is significant.

Figure 6: Paired t-test formula

$$t = \frac{\bar{d} - \mu_0}{\frac{s_d}{\sqrt{n}}} \tag{1}$$

Figure 6 has a standard deviation (s_d) of 0, as the difference between all results was 1. As a result of this, it leads to a division by zero. Depending on interpretation, this could be seen as t approaching infinity (∞) or simply an undefined result.

Under the basis of t approaching infinity, we can consequently view it as a large t value, which suggests there is statistical significance in the results.

5.3.4 Likelihood to support screen reader users

The following figure, namely Figure 7, showcases the results from the Likert scale data obtained in the final questioning (stage 6), which queries participants on their likelihood to

Table 8: Simple aspects from the initial and final listenings

Initial document	Final document
Standalone sentences were read slower and with pauses.	Standalone sentences were read slower and with pauses.
Sentences separated by images or tables were easier to distinguish.	Sentences separated by images or tables were easier to distinguish.
Lists provided in bullet point form were simple to understand due to a clear starting point.	Lists provided in bullet point form were simple to understand due to a clear starting point. Each point in the list started with 'bullet' instead of 'bullet point' which was the case previously, consequently speeding up the screen reader's output.
The table discussing grades is read left to right, whereas the table discussing names is read top to bottom. When the table can be viewed left to right, it followed the same order the screen reader used when creating the audio output.	The table discussing grades is read left to right, whereas the table discussing names is read top to bottom. When the table can be viewed left to right, it followed the same order the screen reader used when creating the audio output.
Supplementary sentences included before or after images and tables improved the overall clarity of the related extracts.	Supplementary sentences included before or after images and tables improved the overall clarity of the related extracts.
	Images were easier to understand due to their included descriptors.
	The screen reader included more pauses and spoke slower, with emphasis placed on tables.
	Structural indicators such as 'out of list', 'out of table', and 'table with M columns and N rows' improved tracking of where the screen reader was on the document at that moment.
	Reading the table heading before each table data cell improved tracaking of what data correlated to what name in the first table.

support screen reader users in the future.

It should be put under consideration that all participants of the user study had indicated a lack of prior experience with screen readers.

5.3.5 LinTEX usability focuses

The following table, namely Table 9, highlights areas where usability could be improved or was found largely useful during the think-aloud session (stage 4). It should be noted that some results that stemmed from a lack of LaTeX knowledge were not included, as Table 9 aimed to identify the key components of usability.

5.4 Analysis

5.4.1 Reasoning behind document extracts being memorable

This section uses data from Table 6 to make an analysis of the reasoning behind why specific sections were memorable, and others were not.

Figure 7: Participants likelihood to support screen reader users in the future

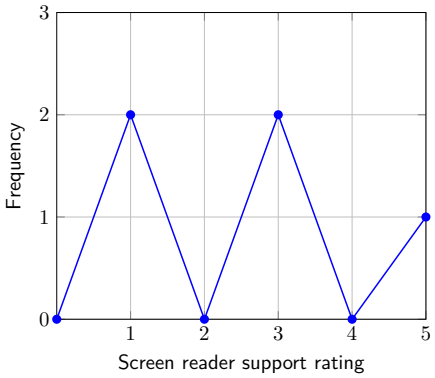


Table 9: LinTEX usability

Affected area	Comment
Uploading a file	Lacked responsiveness regarding actions performed when the upload button was clicked without a selected file, and informing the participant that a file has been uploaded.
Button phrasing	Phrasing of 'generate output' button caused participants to assume it could only be clicked after all errors were fixed.
Scrolling options	Scrolling options for the in-built code editor, errors, and the website as a whole, made navigation more difficult.
Submitting a document	A keybind to generate the document's PDF output was requested.
Error extracts	Most error extracts, the code taken from where the error had occurred, were helpful. However, a few syntax errors, such as a missing closing curly brace only provided the opening curly brace within its extract. It should be noted that many users utilised the browser's search features to locate where the error had occurred.
Image error messages	No difficulties found.
Table error messages	Attempts were made to place the 'tabular' group within the required argument of '\tagpdfsetup'.
Tagging error messages	Uncertainty existed with what should be provided to the required argument of '\DocumentMetadata' and how to handle several key-value pairs being provided to it, with some attempts to add an extra required argument.
Error messages	Examples helped to achieve the correct output, with long error messages found from requesting more details alleviating many prior concerns.
Text size	Text size was small.
Colour scheme	The colour scheme of the website may be inaccessible for users with dyslexia.

While the tongue twister was memorable in the initial listening (stage 2), likely due to it having a relatively large extract focused on it, participants had noted that providing the actual tongue twister made for a more enjoyable and consequently memorable experience in the final listening (stage 5).

Participants frequently misremembered finer details in smaller extracts relating to images, with only the extract on the cat being the favourite pet being the most memorable, likely due to its simplicity. This could be biased by the participants not being informed what specific questions would be asked, or the question being asked after the full document has been read, consequently requiring the participants to keep the full document in mind by

that point. The latter point highlights a struggle a screen reader user may face.

Regarding name positioning and grade comparisons, a reasonable assumption as to why the overall topics were memorable despite finer details being frequently incorrect is that they required a greater degree of focus due to having more data in the tables. However, due to the magnitude of the data, and the speed and lack of pauses in their listening, as referenced in Table 7, the particulars of the data were often mistaken. Additionally, supplementary sentences surrounding these tables often provided a general overview of what the tables had contained, which may have influenced a participant's understanding.

5.4.2 LinTEX's impact on document screen reader accessibility

Figure 5 showcases that the accessible document's audio output rating had a higher peak compared to the inaccessible document. Additionally, we can view that the accessible document is shifted one data point higher on the x-axis at all points, with the same overall shape maintained as the inaccessible document. We can consequently conclude that, based on this quantitative data, LinTEX had provided an improvement to the document's accessibility, with emphasis placed on screen readers.

Table 7 highlights the components of the initial listening (stage 2) and final listening (stage 5) that were difficult to understand. This permits a comparison between the two columns, for which we can determine that the majority of difficult aspects had been improved, as they were no longer in the final listening (stage 5)'s column. However, there remained two components that provided some confusion in the final listening (stage 5):

- The document's contents added unnecessary confusion at times, where it could be confused for metadata that the screen reader was reading out.
- The tables within the document contained too much metadata. The phrasing 'less is more' could be applied here, where too much information obstructed a participant's ability to understand its contents.

Table 8 showcases the fact that there were many more simple to understand components in the accessible document compared to the inaccessible document, with many of the previously helpful components remaining the same or having mild improvements.

Consequently, we can deduce that simple components were retained with the same, or arguably better, quality.

5.4.3 Areas of improvement for LinTEX usability

Table 9 highlights many areas where LinTEX suffers in its usability. Hence, some proposed solutions for these concerns have been indicated below:

- The file upload process could be simplified by combining both the file selector and the upload file button. This would reduce the number of steps required on the user's part to upload the file and begin using LinTEX.
- The phrasing of specific website components could be changed. For example, 'generate output' could be swapped out for 'check document' which has different connotations of applying some form of validation to the document. Additionally, a keybind to shortcut clicking this button has been indicated to be a desired feature, due to the reliance on frequently clicking it.
- Scrolling within the webpage could be alleviated by removing scrolling of the full webpage (for the tool webpage). That would ensure there is always a 'check document' button at the bottom, while still allowing the flexibility of scrolling error messages or the code editor.
- More information should be provided by some error messages, with emphasis on those related to tables or tagging. By providing more information upfront, there will be less confusion about where to progress from a specific point.
- The error extracts being less helpful in the context of syntax errors should be alleviated by providing the line and character position information. In the future, if there is enough time, an automated fix feature could be implemented.
- Text sizes being too small can be solved by simply increasing their size. There is also potential for choosing different font families that fit the context better. Some consideration should be put towards whether the browser should be relied on for resizing of elements, rather than LinTEX implementing it.
- The colour scheme was indicated to have not been distinct enough for dyslexia users,

which can be alleviated by choosing and applying a more accessible colour scheme for this user group.

5.5 Hypotheses testing and discussion

It can be claimed that, as there is statistical significance in applying student's paired t-test (section 5.3.3), a right-tailed test can be used to ascertain that there is a definitive improvement in the screen reader accessibility of the documents after applying LinTEX. Additionally, based on section 5.3.2, it can be concluded that there was a significant improvement in the document's screen reader accessibility after applying LinTEX, as it showed previously difficult aspects becoming simpler to understand, with minimal difficult aspects in the accessible document. Consequently, we choose not to reject the hypothesis **H1**.

The hypothesis **H2** is a grey area, due to the magnitude of accessibility-related difficulties found in the Table 9. However, proposed solutions have been provided in areas of improvement for LinTEX usability (section 5.4.3), which indicate processes that may be undertaken to solve many of the identified concerns. As many of these solutions are resolvable with little effort, it suggests the accessibility concerns LinTEX faces are minor.

It can be reasonably chosen to not reject the hypothesis **H3** as the majority of error messages were greatly useful based on Table 5.4.3, with only a few points made to provide more information upfront to avoid the user being at a standstill for what action to perform at a given point and providing more information regarding the location the error had occurred.

The hypothesis **H4** is arguable, as the main key points were often recalled by the participants. However, as many finer details were significantly wrong, the overall memorability of the inaccessible document's content could be considered low. Consequently, this would permit not rejecting the hypothesis **H4**.

6 Discussion

6.1 Objectives discussion

OBJ1 has been met, as the depths of research for the technologies used (section 4.1), design pattern considerations in section 3.4, and documentation (section 3.6), make for a high likelihood for a maintainable product.

As the hypothesis **H1** was not rejected, it can be reasonably assumed the same for the objective **OBJ2**.

The hypothesis, **H3**, was not rejected. This suggests the objective **OBJ3**. Additionally, the depth of work put into the error handling (section 3.5) makes it certain that a substantial amount of error handling was provided to the end user.

Lastly, it is considered a slight grey area whether LinTEX was accessible, as indicated in hypotheses testing and discussion (section 5.5) with hypothesis **H2** having a strong correlation with the objective **OBJ4**.

6.2 Research question discussion

RQ1 has the key considerations, for its application in LATEX, noted in LinTEX supported accessibility (section 3.3). We can consider these to have been accurate identifications for LATEX acceessibility for screen reader users, as the objective **OBJ2** was largely a success.

RQ2 was discussed in the introduction (section 1). We may insinuate it has been confirmed through the success of **RQ1**, as it suggests the reasonings initially found are correct if the solutions are reasonably valid.

7 Conclusions

7.1 Project summary

After investigating causes for inaccessible LaTeX document for screen reader users, including misinformation and outdated PDFLaTeX usage, several solutions were identified in LinTeX supported accessibility (section 3.3), leading to the creation of LinTeX.

LinTeX was developed to target key screen reader-focused acceessibility concerns for LaTeX users, namely tagging and alternative text. Additionally, it aimed to be a maintainable and accessible tool, to ensure its use by many users and its longevity to keep up with frequently changing demands for accessibility.

The results from the user study highlight the project succeeded in making an inaccessible document accessible for screen reader users, with some mild nuances in the usability of the tool. Due to the depth of research in related topics, LinTeX was made to be especially maintainable.

7.2 Project limitations

In an ideal world, it would have been possible to have created a project that could target any and all LaTeX accessibility concerns. Unfortunately, that is outwith the scope of the project, which has been defined as follows:

- LinTeX is not designed to handle the majority of parsing. Instead, it focuses on a few key elements that are related to the accessibility concerns targetted in our context. Otherwise, it handles a small amount of verification regarding the structure of individual lines and the overall document structure.
- This project does not aim to provide automated fixes for errors encountered.
- LinTeX does not contain an in-built screen reader. While some considerations were put aside for how this could be accomplished, it is not feasible to create an in-built screen reader with the timeframe allocated for this project, due to the vast considerations needed. It might even be a fair statement to consider it a project on its own.
- LinTeX is not intended to find usage of outdated or incorrect accessibility, but rather to

enhance a document's screen reader accessibility through additions to the document.

- LinTEX is not intended to debug LATEX code, as a dedicated integrated development environment (IDE) and TEX engine should be used for this instead.
- LinTEX expects code to compile in LuaLATEX before its changes have been applied, with other TEX engines being considered outdated and consequently not supported for this context.
- LinTEX places emphasis on accessibility features for screen reader users.
- It is unfeasible to account for all tagging options in our context. Therefore, a few key options have been selected based on what would be expected to have the most impact.
- It is outwith the scope of this project to consider the security implications of local LATEX compilation.

7.3 Achievements

LinTEX achieved many targets, including but not limited to:

- Implementation of several design patterns and documentation, as indicated in section 3.4 and section 3.6 respectively, ensuring the project's maintainability.
- Successfuly uploading a LATEX file, previewing the generated PDF file, and downloading it.
- Providing in-depth error messages with several changes depending on their context, as indicated in error handling (section 3.5).
- Successfully making an inaccessible document support tagging and image descriptors.
- Creating a tool, LinTEX, which is very quick on launch and takes minimal time to load documents.

7.4 Future work

LinTEX has the potential to grow in a variety of manners, such as targetting the limitations in project limitations (section 7.2).

An in-built screen reader for LinTEX is an interesting proposition, with it likely being a project by itself due to the lack of finer details requiring vast amounts of consideration and the difficulties with finding similar materials, seeing as most references for an in-built screen reader appear to point to screen readers as a browser extension. However, it may be possible to make LinTEX work with those extensions to provide the ideal experience for the end user.

Automated fixes, with the addition of being able to fix several identical errors at once, would be a great feature to include in LinTEX. There is potential for this feature to work alongside the error handling of LinTEX, yet this has unfortunately not been explored further at this time.

Additionally, highlighting why LinTEX has recommended a change would make the experience a lot more insightful and better encourage developers to be inclusive of screen reader users in the future.

Finally, utilising HTMX to make LinTEX more responsive to the end user's actions would allow for a more immersive experience for the user, while keeping the technologies required at a reasonable minimum requirements.

7.5 Reflection

At the start of this project, I was overzealous with all the targets I had in mind for it. However, this led to research, time and energy spent in areas that ended up unused such as HTMX, the University of Strathclyde's DevWeb servers, automated error fixes, and the potential for an in-built screen reader.

The time and energy spent on those enthusiastic tasks could have been spent on a simpler, yet just as advantageous, task. For example, researching methods in which accessibility is incorrectly used in PDFLATEX and identifying those examples, to promote more accessible alternatives.

7.6 Final thoughts

Despite the hardships in creating LinTEX, it has been an enjoyable process with many quirks paved through the road. There are a lot of methods to expand on this project, as indicated

in future work (section 7.4), which would be fascinating to see the results for.

As a final note: avoid PDFLaTeX ; use LuaLaTeX instead.

References

Angers, Martin (2025). *pigeon - a PEG parser generator for Go*. English. https://pkg.go.dev/github.com/mna/pigeon. URL: https://pkg.go.dev/github.com/mna/pigeon.

Authors, The go-latex (2020). *latex*. English. hhttps://pkg.go.dev/codeberg.org/go-latex/latex. URL: https://pkg.go.dev/codeberg.org/go-latex/latex.

Authors, The Bootstrap (2011). *Bootstrap*. English. https://getbootstrap.com/. URL: https://getbootstrap.com/.

Authors, The Go (2009a). *Exec*. English. https://pkg.go.dev/os/exec. URL: https://pkg.go.dev/os/exec.

– (2009b). *html/template*. English. https://pkg.go.dev/html/template. URL: https://pkg.go.dev/html/template.

– (2009c). *net/http*. English. https://pkg.go.dev/net/http. URL: https://pkg.go.dev/net/http.

Authors, The Gorilla (2023). *gorilla/mux*. English. https://pkg.go.dev/github.com/gorilla/mux. URL: https://pkg.go.dev/github.com/gorilla/mux.

Center, Delaware Government Information (2024). *What is User Research and Why Should it be a Part of Your Process?* English. URL: https://gic.delaware.gov/what-is-user-research-and-why-should-it-be-a-part-of-your-process/.

Clifton, Andy (2019). *The Accessibility LaTeX package*. English. https://github.com/AndyClifton/accessibility. URL: https://github.com/AndyClifton/accessibility.

Doctor, Nathan **and** Jake Hoffner (2012). *CodeWars*. English. https://www.codewars.com/. URL: https://www.codewars.com/.

Exeter, University of (2025). *Test Statistics*. English. URL: https://libguides.exeter.ac.uk/c.php?g=729256&p=5313318.

Inc., Bergside (2023). *Flowbite*. English. https://flowbite.com/. URL: https://flowbite.com/.

Kpym (2020). *latex-fast-compile*. English. https://pkg.go.dev/github.com/kpym/latex-fast-compile. URL: https://pkg.go.dev/github.com/kpym/latex-fast-compile.

Martínez-Almeida, Manuel (2014). *Gin Web Framework*. English. https://pkg.go.dev/github.com/gin-gonic/gin. URL: https://pkg.go.dev/github.com/gin-gonic/gin.

Murrish, Robert (2012). **?**LaTeX is More Powerful than you Think - Computing the Fibonacci Numbers and Turing Completeness**?** English. **in**URL: https://www.overleaf.com/learn/latex/Articles/LaTeX_is_More_Powerful_than_you_Think_-_Computing_the_Fibonacci_Numbers_and_Turing_Completeness.

National Library of Medicine, Network of the (2026a). *Qualitative Data*. English. URL: https://www.nnlm.gov/resources/data-glossary/qualitative-data.

– (2026b). *Quantitative Data*. English. URL: https://www.nnlm.gov/resources/data-glossary/quantitative-data.

Noback, Matthias (**january** 2018). **?**The Dependency Inversion Principle: Creating Reusable Software Components**? inpages** 67–104: ISBN: 978-1-4842-4118-9. DOI: 10.1007/978-1-4842-4119-6_5.

Ravisankar, Vivek **and** Harishankaran Karunanidhi (2011). *HackerRank*. English. https://www.hackerrank.com/. URL: https://www.hackerrank.com/.

Shvets, Alexander (2014a). *Adapter, also known as: Wrapper*. English. URL: https://refactoring.guru/design-patterns/adapter.

– (2014b). *Builder*. English. URL: https://refactoring.guru/design-patterns/builder.

– (2014c). *Composite, also known as: Object Tree*. English. URL: https://refactoring.guru/design-patterns/composite.

Software, Big Sky (2020). *HTMX*. English. https://htmx.org/. URL: https://htmx.org/.

Sullivan, Gail M. (2013). **?**Analyzing and Interpreting Data From Likert-Type Scales**?** English. **in**URL: https://pmc.ncbi.nlm.nih.gov/articles/PMC3886444/.

Tailwind Labs, Inc. (2017). *Tailwind CSS*. English. https://tailwindcss.com/. URL: https://tailwindcss.com/.

Tang, Winston (2015). *LeetCode*. English. https://leetcode.com/. URL: https://leetcode.com/.

University, Ulster (2026). *What is accessibility and why is it important?* English. URL: https://www.ulster.ac.uk/accessibility-standards/what.

W3C (2018). *Web Content Accessibility Guidelines (WCAG) 2.2*. https://www.w3.org/TR/WCAG22/. A W3C Recommendation. URL: https://www.w3.org/TR/WCAG22/.

Wadhwa, Raoul R. **and** Raghavendra Marappa-Ganeshan (2023). **?**T Test**?** English. **in**URL: https://www.ncbi.nlm.nih.gov/books/NBK553048/.

A Appendix

A.1 Ethical approval

University of Strathclyde Glasgow

Computer and Information Sciences - local.cis

departmental information for staff and students

CIS Ethics Approval System

You are Mohammad Danyal (CS2022 - 202318141)

Background

The purpose of the Departmental and University Ethics Committees is to ensure that any investigations (regardless of subject domain) carried out by staff, or students that use human beings as participants are known to conform to the standards set by the professional bodies. These standards are aimed at ensuring that the rights, safety, and wellbeing ofthe participants is taken into account at all times. The Departmental Ethics Committee members are Andrew Abel, Daniel Thomas, Emma Nicol, John Connaghan, Karen Renaud, Lisa McCann, Louise McCullagh, Marc Roper, Mark Dunlop, Martin Halvey, Morven Miller, Pejman Saeghe, Ryan Gibson, and Yingying Zhao. You may contact the committee using the following email address: cis-ethics@strath.ac.uk

What you need to do

It is important to realise that any investigation that involves human beings as participants - even something as apparently harmless as a questionnaire or simple user study being undertaken as part of a project or your research - has to be granted ethical approval. To gain this approval you need to complete the "New application" process at the end of this page and upload supporting documentation which addresses the following points:

- Title of research:
- Summary of research (short overview of the background and aims of this study):
- How will participants be recruited?
- What will the participants be told about the proposed research study?
  Either upload or include a copy of the briefing notes issued to participants. In particular this should include details of yourself, the context of the study and an overview of the data that you plan to collect, your supervisor, and contact details for the Departmental Ethics Committee.
- How will consent be demonstrated?
  Either upload or include here a copy of the consent form/instructions issued to participants. It is particularly important that you make the rights of the participants to freely withdraw from the study at any point (if they begin to feel stressed for example), nor feel under any pressure or obligation to complete the study, answer any particular question, or undertake any particular task. Their rights regarding associated data collected should also be made explicit.
- What will participants be expected to do?
  Either upload or include a copy of the instructions issued to participants along with a copy of or link to the survey, interview script or task description you intend to carry out. Please also confirm (where appropriate) that your supervisor has seen and approved both your planned study and this associated ethics application.
- What data will be collected and how will it be captured and stored?
  In particular indicate how adherence to the Data Protection Act and the General Data Protection Regulation (GDPR) - https://en.wikipedia.org/wiki/General_Data_Protection_Regulation - will be guaranteed and how participant confidentiality will be handled.
- How will the data be processed?
  (e.g. analysed, reported, visualised, integrated with other data, etc.) Please pay particular attention to descibing how personal or sensitive data will be handled and how GDPR regulations will be met.
- How and when will data be managed or disposed of?
  Either upload a copy of your data management plan or describe how data will be disposed.

The form should be submitted well in advance of any planned study. Requests are normally processed within one working week, but may take longer than this for more complex studies or at certain times of the year.

The points above cover most studies carried out within the Department, but if you are in any doubt then you should read the University's Code of Practice on Investigations on Human Beings to make sure that there are not any other issues you should be considering. If you are uncertain about any aspect of your study then discuss this initially with your supervisor but also feel free to contact the Departmental Ethics Committee. More significant studies may need to be referred to the University Ethics Committee (UEC) and are likely to take much longer to process.

For the participant information sheets and consent form, you may find the pro-formas available from the UEC page useful https://www.strath.ac.uk/ethics/ However, please take care to amend these appropriately and pay particular attention to the approval and contact details.

Contact Details

email:cis-ethics@strath.ac.uk

New application

ID	TITLE	STATUS	ACTION	RELATED IDS	LAST SUBMITTED	LAST REVIEWED
3188	LinTeX: Promoting Accessibility in LaTeX Documents	APPROVED	View | Amend		2026-03-04 03:38:29	2026-03-09 12:49:45

A.2 Ethics application

CIS Ethics Approval System

You are Mohammad Danyal (CS2022 - 202318141)

Return to Main

Application ID: 3188

Title of research:
LinTeX: Promoting Accessibility in LaTeX Documents

Summary of research (short overview of the background and aims of this study):
LinTeX is a new website being developed to combat accessibility problems in LaTeX, with a focus on screen readers. This study aims to understand its effectiveness at targeting this problem and highlighting solutions to its users. Additionally, it aims to identify areas for improvement.

How will participants be recruited?
Participants will be recruited via email and the 4th Year CompSci Strath 2025/26 Discord server in a non-invasive manner. The following message will be used in both contexts:
“Dear <Participant>,

I would like your views on a tool I'm making that helps people write better LaTeX documents. In particular, it helps authors make documents that meet accessibility needs, and it helps authors with accessibility needs write their LaTeX documents.

LinTeX is a tool being developed to combat this dilemma, placing emphasis on accessibility for screen readers. This user study will determine its effectiveness in doing so.

This user study will involve an in-person think-aloud session taking place on the University of Strathclyde’s campus. Further details can be found in the Participant Information Sheet (PIS) attached.

To participate in the user study, you should be a member of the University of Strathclyde, or another higher education facility. Some prior experience with LaTeX will be expected, and participants should be 18-years old or above.

Participation is entirely voluntary and will not affect you in any courses or how you are treated. If you could spare around 30 minutes to participate in the user study, it would be greatly appreciated.

If you are interested in participating in the user study, please contact me via email.

My contact details:
Mohammad Danyal
mohammad.danyal.2023@uni.strath.ac.uk

My supervisor’s details:
Brian Mitchell
brian.mitchell@strath.ac.uk

Kind regards,
Mohammad Danyal”

- ‘<Participant>’ will be replaced with the group/individual being contacted.

What will the participants be told about the proposed research study? Either upload or include a copy of the briefing notes issued to participants. In particular this should include details of yourself, the context of the study and an overview of the data that you plan to collect, your supervisor, and contact details for the Departmental Ethics Committee.
PDF File: View document
Details listed on Participant Information Sheet (PIS).

How will consent be demonstrated? Either upload or include here a copy of the consent form/instructions issued to participants. It is particularly important that you make the rights of the participants to freely withdraw from the study at any point (if they begin to feel stressed for example), nor feel under any pressure or obligation to complete the study, answer any particular question, or undertake any particular task. Their rights regarding associated data collected should also be made explicit.
PDF File: View document

Details listed on Consent Form.

What will participants be expected to do? Either upload or include a copy of the instructions issued to participants along with a copy of or link to the survey, interview script or task description you intend to carry out.
PDF File: View document
PDF File: None.
Details listed on the User Study Script.

What data will be collected and how will it be captured and stored? In particular indicate how adherence to the Data Protection Act and the General Data Protection Regulation (GDPR) will be guaranteed and how participant confidentiality will be handled.
Several pieces of data will be stored:
1. Assigning participants unique IDs. This section will have information such as their name and email address (to contact the participant for arranging a time/date). Participant IDs will be created when the participant has requested to join the user study.
2. Extracting details from the user study (such as answers to questions, audible points made by the participant, or unspoken points of note such as how long a specific task has taken to complete) and assigning it to the relevant participants' ID, consequently ensuring related data can be deleted as requested. All details for this group will be acquired from the user study.
3. Anonymising the data from the user study to be used in the final report. This section will have no participant IDs and will have no relation to any participant of the user study. Data will be anonymised at the earliest practicable stage.

Each group of data, as numbered above, will be stored only in separate folders on the university's OneDrive, within password-protected folders, ensuring a folder for each numbered section.

How will the data be processed? (e.g. analysed, reported, visualised, integrated with other data, etc.) Please pay particular attention to descibing how personal or sensitive data will be handled and how GDPR regulations will be met.
Anonymised data will be inserted into the final project report after being thoroughly analysed.
Some examples may include:
- How often a listening, or a section of the listening, has been repeated at a stage;
- How often and where a section of the listening has been paused;
- Comparing answers with questions from the initial hearing to the hearing after having used the LinTeX tool;
- Analysing how well the overall document was understood initially to determine difficulties with screen readers;
- Analysing correlations between a user's experience with LaTeX and screen readers with the results from the other components of the user study;
- Mathematical analysis of Likert scale data including frequencies, median, percentages, and mode;
- Comparison of Likert scale data from before and after the think-aloud session.

Results will be presented to minimise the chance of any participant being identifiable. Additionally, any qualitative results will be anonymised.

How and when will data be disposed of? Either upload a copy of your data management plan or describe how data will be disposed.
PDF File: None.
Physical notes taken during the user study will be digitalised as soon as reasonably possible. Once digitalised, they will be securely destroyed.
Electronic data will be securely destroyed on 1st August 2026.

Supervisor's name (students need to complete/staff should probably put N/A).
Brian Mitchell

☑ I confirm that my supervisor has seen and approved both my planned study and this associated ethics application.

A.3 Participant information sheet

Participant Information Sheet for LinTeX User Study

[FOR USE WITH STANDARD PRIVACY NOTICE FOR RESEARCH PARTICIPANTS]

Name of department: Computer and Information Sciences
Title of the study: LinTeX: Promoting Accessibility in LaTeX Documents

Introduction

I am Mohammad Danyal, a final year Undergraduate Computer Science student at the University of Strathclyde. I can be contacted by email via mohammad.danyal.2023@uni.strath.ac.uk should any concerns or questions arise regarding the study, or if you wish to withdraw your participation.

What is the purpose of this research?

Modern LaTeX documents tend to use PDFLaTeX as opposed to LuaLaTeX. Thus, they often require workarounds to implement features and lack accessibility support.
LinTeX is a prototype to combat accessibility with emphasis placed on screen readers, and avoiding counter-productive workarounds.
This user study will identify the effectiveness of a new tool, LinTeX, to promote accessibility in LaTeX documents, as well as its usability to a wider audience.

Do you have to take part?

Participation in the research is entirely voluntary.
Refusing to participate in or withdrawing from the study will not affect the ways an individual is treated.
Participants may withdraw from the study at any stage without providing a reason.
If you wish to withdraw from the study at any stage, please contact mohammad.danyal.2023@uni.strath.ac.uk.

What will you do in the project?

Participants will be asked to meet in-person in the University of Strathclyde's campus at a pre-arranged time and date.
Participants will be provided with a LaTeX document at the start of the user study. Participants will then listen to a screen reader's output for the LaTeX document.
Afterwards, participants will be asked questions about their thoughts on the output.
Participants will then be requested to use LinTeX in a think-aloud session, where you say what you are thinking, to improve the document's accessibility through improvements suggested by LinTeX.
Participants will finally repeat the initial step of listening to a screen reader's output for the improved document and answer questions regarding their opinion of what went well and what could have been improved.

Why have you been invited to take part?

Participants of the research study may benefit from the LinTeX tool and have been asked to assist in its development through an in-person think-aloud user study.
Participants must be a member of the University of Strathclyde, or another higher education facility.
Travel costs cannot be reimbursed.
Participants should be 18-years old or above.

Participants will require prior experience with LaTeX to participate in the study and have been selected based on these criteria.

What information is being collected in the project?
Personal information that may be stored include the participants' names and email addresses. This datum is stored to ensure communication is thorough in arranging a time and date to participate in the study. Additionally, some information stored about the participants' usage of the system or answers to questions may be personal.
Non-identifiable information that may be stored include anonymised information extracted from the user study.
No audio or video recording will be created. No transcripts will be created.

Who will have access to the information?
Only the researcher will have access to any Personally Identifiable Information (PIP). Anonymised information may be included in the research paper and shared with the supervisor.

Where will the information be stored and how long will it be kept for?
All data will be stored only in password-protected folders on OneDrive, with Personally Identifiable Information (PIP) and anonymous data stored in separate folders.
All data stored in these folders will be deleted on 1st August 2026. However, any data that is anonymised and included in the study may not be possible to remove.
Physical notes may be taken during the user study, which will be digitalised within a week. Once digitalised, it will be securely destroyed.
Thank you for reading this information – please ask any questions if you are unsure about what is written here.
All personal data will be processed in accordance with applicable Scottish data protection legislation. Please read our Privacy Notice for Research Participants for more information about your rights under the legislation.

What happens next?
If you are interested in the project, or continuing with the user study, please contact me by email via mohammad.danyal.2023@uni.strath.ac.uk. Participants will be requested to sign a consent form prior to participation in the user study.
All information collected in the user study will be anonymised before being published.
If participants are interested in participating in any future related studies, the aforementioned email address can be used to register your interest.
If you are no longer interested in participating in the project, no further steps will be required. Thank you for your time.

Researcher contact details:
Mohammad Danyal
mohammad.danyal.2023@uni.strath.ac.uk

Chief Investigator details:
Brian Mitchell
brian.mitchell@strath.ac.uk

This research was granted ethical approval by the Department of Computer and Information Sciences Ethics Committee.

If you have any questions/concerns, during or after the research, or wish to contact an independent person to whom any questions may be directed or further information may be sought from, please contact: cis-ethics@strath.ac.uk

A.4 Consent form

Consent Form for LinTeX User Study

Name of department: Computer and Information Sciences
Title of the study: LinTeX: Promoting Accessibility in LaTeX Documents

Please check each item to indicate agreement.

- ☐ I confirm that I have read and understood the Participant Information Sheet for the above project and the researcher has answered any queries to my satisfaction.
- ☐ I confirm that I have read and understood the Privacy Notice for Participants in Research Projects and understand how my personal information will be used and what will happen to it (i.e. how it will be stored and for how long).
- ☐ I understand that my participation is voluntary and that I am free to withdraw from the project at any time, up to the point of completion, without having to give a reason and without any consequences.
- ☐ I understand that I can request the withdrawal from the study of some personal information and that whenever possible researchers will comply with my request. This includes the following personal data:
    - My personal information collected from the think-aloud session.
    - My personal information from answers to questions.
    - My personal information from transcripts.
- ☐ I understand that anonymised data (i.e. data that do not identify me personally) cannot be withdrawn once they have been included in the study.
- ☐ I understand that any information recorded in the research will remain confidential and no information that identifies me will be made publicly available.
- ☐ I consent to being a participant in the project.

(PRINT NAME)	
Signature of Participant:	Date:

A.5 User study script

Script for LinTeX User Study

Name of department: Computer and Information Sciences
Title of the study: LinTeX: Promoting Accessibility in LaTeX Documents

Introduction

Hello, I am Mohammad Danyal, a final year Computer Science student at the University of Strathclyde. I am the Researcher for this user study.

As a general overview of what this user study will consist of:

- Introductory questions to gauge experience levels
- An initial phase for listening to the document through a screen reader.
- Answering questions about how easy or difficult the document is to understand.
- Utilising the proposed tool, LinTeX, to make the document more accessible to screen reader users.
- A final listening of the document, with the changes requested by the tool having been applied.
- Answering further questions about whether the tool had made an impact on the difficulty for understanding the document as a screen reader user.

These tasks will be explained in more detail as we approach them.

If you have any questions as we go through the process, feel free to chime in.

You have the right to opt to not answer questions as you wish, as stated in the consent form.

Introductory questions

- On a scale of 1 to 5, how often do you use LaTeX?
    1. Never
    2. Rarely
    3. Sometimes
    4. Often
    5. Almost always
- On a scale of 1 to 5, how often do you use screen readers?
    1. Never
    2. Rarely
    3. Sometimes
    4. Often
    5. Almost always
- On a scale of 1 to 5, how familiar are you with listening to audio recordings of speaking?
    1. Very unfamiliar
    2. Unfamiliar
    3. Neither unfamiliar/familiar
    4. Familiar
    5. Very familiar

Initial listening

For this step, I will have you listen to the output of the document before any changes have been made to it. If you have any device for listening to the output that you'd prefer to use, just make that preference known and it can be setup.

While listening to the output, if you would like to make any points of interest, simply indicate that and the audio output can be paused. Additionally, if you would like to repeat any sections or the full recording, feel free to ask.
When the recording has been complete, and no requests are made to repeat a section, we will move onto answering a few questions.

Questions [initial listening]
- Provide a general overview of what the document discussed.
- What parts of the listening were simple to understand? Why?
- What parts of the listening were difficult to understand? Why?
- On a scale of 1 to 5, how would you rate the quality of the audio output? (i.e. in terms of the content of its output)
    1. Very poor
    2. Poor
    3. Neither poor/good
    4. Good
    5. Very good
- How well could you understand the purpose of tables, images or figures in the listening?

Using LinTeX
In this step, we will perform a think aloud session. In case you have not participated in one before, here is a general summary:
- You will be tasked to perform some action.
- You should audibly state your thought process as you go through the task. For example, this may be in the form of the stating having trouble with something.
- There will be minimal intervention from me.

The task is in 3 parts:
1. Upload the document to LinTeX by navigating its GUI
2. Improve the document's accessibility by following what LinTeX provides.
3. Export the document.

You will be provided with the file's location, as well as the LinTeX tool to work with.

Final listening
Similar to before, you will be tasked with listening to the output of the document. This time, after changes have been applied through the LinTeX tool.
If you would like to make any points of interest, simply indicate that and the audio output can be paused. Additionally, if you would like to repeat any sections or the full recording, feel free to ask.
When the recording has been complete, and no requests are made to repeat a section, we will move on to answering the final questions.

Questions [final listening]
- What parts of the listening were simple to understand? Why?
- What parts of the listening were difficult to understand? Why?

- On a scale of 1 to 5, how would you rate the quality of the audio output? (i.e. in terms of the content of its output)
    1. Very poor
    2. Poor
    3. Neither poor/good
    4. Good
    5. Very good
- How well could you understand the purpose of tables, images or figures in the listening?
- On a scale of 1 to 5, how likely are you accommodate for screen reader users in the future?
    1. Very unlikely
    2. Unlikely
    3. Neither unlikely/likely
    4. Likely
    5. Very likely

Exit

Thank you for your time. You have been of great help for this user study. If you wish to see where the project may end up or have any questions, my email is on the Participant Information Sheet under the Researcher.

A.6 Inaccessible document

```
\documentclass[draft]{article}
\usepackage{graphicx}
\begin{document}
\section{Introduction}
This document is intended to be read by a screen reader to indicate related
accessibility concerns for LaTeX users.

\section{Image Examples}
We will have 3 examples for our context:
\begin{itemize}
      \item An example of an image that should contain alternative text ('alt
text'). Alternative text is a descriptive text used in place of an image or
graphic.
      \item An example of an image that has actual text within it that must be
read
      \item An example of an image that is purely for decorative purposes i.e.
it should not be read.
\end{itemize}
These will be in no particular order. Some filler text will be included to make
them appear more natural.

Every day for a year, I have practiced my speech. I have used many techniques to
improve my speaking skills, such as socialising, taking lessons, trying out
tongue twisters, amongst many other things.\\
\includegraphics{tongue_twister.jpg}\\ % use actual text
That was an example of a tongue twister I have mastered.

This house is for sale. Its market price is currently \$100,000, but it is
estimated to rise to \$250,000 in a year.\\
\includegraphics{simple_house.png}\\ % use alt text

I have many pets, but my favourite is my cat. Cats are great!\\
\includegraphics{cat.png}\\ % use decorative (artifact) or alt text

\section{Table Examples}
We will have 2 examples for our context:
\begin{itemize}
      \item An example of a table that should have its heading highlighted.
      \item An example of a table that is purely for decorative purposes i.e. it
should not be read.
\end{itemize}
These will be in no particular order. Some filler text will be included to make
them appear more natural.

Names are interesting. How much more common is the name Noah compared to
Muhammad in Scotland?\\
\begin{tabular}{ccc} % spoken table
      Year & Muhammad Position & Noah Position \\\hline
      2024 & 2nd & 1st \\
      2023 & 11th & 2nd \\
      2022 & 23rd & 1st \\
      2021 & 43rd & 2nd \\
      2020 & 31st & 2nd \\
\end{tabular}\\
From this table, we can see that originally Muhammad was a far less common name
than Noah, but in the period of 5 years Muhammad has drastically rose from 31st
place to 2nd place, making it a strong competitor against Noah which has
retained a top 2 placing across all years.

Here are my grades from this year, compared to my friends' grades.\\
\begin{tabular}{ccccc} %  presentation table
      Name & CS913 & ML123 & MS019 & HI129 \\\hline
      James & 12\% & 20\% & 80\% & 67\% \\
      Patricia & 80\% & 90\% & 31\% & 19\% \\
```

```
      John & 34\% & 89\% & 55\% & 98\%
\end{tabular}\\
I think John did the best out of them!
\end{document}
```

A.7 Accessible document

```
\DocumentMetadata{
    tagging = on,
    tagging-setup = {activate/all},
    pdfstandard = ua-2,
    lang = en-GB,
    pdfversion = 2.0
}
\documentclass[draft]{article}
\usepackage{graphicx}
\begin{document}
\section{Introduction}
This document is intended to be read by a screen reader to indicate related
accessibility concerns for LaTeX users.

\section{Image Examples}
We will have 3 examples for our context:
\begin{itemize}
      \item An example of an image that should contain alternative text ('alt
text'). Alternative text is a descriptive text used in place of an image or
graphic.
      \item An example of an image that has actual text within it that must be
read
      \item An example of an image that is purely for decorative purposes i.e.
it should not be read.
\end{itemize}
These will be in no particular order. Some filler text will be included to make
them appear more natural.


Every day for a year, I have practiced my speech. I have used many techniques to
improve my speaking skills, such as socialising, taking lessons, trying out
tongue twisters, amongst many other things.\\
\includegraphics[actualtext={I slit the sheet, the sheet I slit, and on the
slitted sheet I sit}]{tongue_twister.jpg}\\ % use actual text
That was an example of a tongue twister I have mastered.

This house is for sale. Its market price is currently \$100,000, but it is
estimated to rise to \$250,000 in a year.\\
\includegraphics[alt={An image of the house being sold.}]{simple_house.png}\\ %
use alt text

I have many pets, but my favourite is my cat. Cats are great!\\
\includegraphics[artifact]{cat.png}\\ % use decorative (artifact) or alt text

\section{Table Examples}
We will have 2 examples for our context:
\begin{itemize}
      \item An example of a table that should have its heading highlighted.
      \item An example of a table that is purely for decorative purposes i.e. it
should not be read.
\end{itemize}
These will be in no particular order. Some filler text will be included to make
them appear more natural.

Names are interesting. How much more common is the name Noah compared to
Muhammad in Scotland?\\
\tagpdfsetup{table/header-rows={1}}
\begin{tabular}{ccc} % spoken table
      Year & Muhammad Position & Noah Position \\\hline
      2024 & 2nd & 1st \\
      2023 & 11th & 2nd \\
      2022 & 23rd & 1st \\
      2021 & 43rd & 2nd \\
      2020 & 31st & 2nd \\
```

```
\end{tabular}\\
From this table, we can see that originally Muhammad was a far less common name
than Noah, but in the period of 5 years Muhammad has drastically rose from 31st
place to 2nd place, making it a strong competitor against Noah which has
retained a top 2 placing across all years.

Here are my grades from this year, compared to my friends' grades.\\
\tagpdfsetup{table/tagging=presentation}
\begin{tabular}{ccccc} % presentation table
      Name & CS913 & ML123 & MS019 & HI129 \\\hline
      James & 12\% & 20\% & 80\% & 67\% \\
      Patricia & 80\% & 90\% & 31\% & 19\% \\
      John & 34\% & 89\% & 55\% & 98\%
\end{tabular}\\
I think John did the best out of them!
\end{document}
```

A.8 PEG script

```
{
    package main

    var groupStack Stack[Component] = newStack[Component]()
    var documentBuilder IDocumentBuilder = newDocumentBuilder()
    var argumentStack Stack[string] = newStack[string]()
}

/**
 * This section deals with the overall formatting of the document.
 * This includes enforcing an expected format is used, and handling of every
component of the document.
 * Additionally, it is necessary to return the overall Document struct created
to represent the parsed document.
 */
// Matches against overall document structure
Input <- &{
    // re-initialise data
    groupStack = newStack[Component]()
    documentBuilder = newDocumentBuilder()
    argumentStack = newStack[string]()
    return true, nil
} text prerequisite:PrerequisiteContent text docclass:DocumentClass text
preamble:Preamble text content:DocumentContent text EOF {
    // creating coordinates for potential error handling
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }

    // handling prerequisite
    if prerequisite != nil {
        for _, component := range prerequisite.([]Component) {
            errFn := documentBuilder.addPrerequisiteLine(component)
            if errFn != nil {
                return nil, errFn(*startCoord, *endCoord)
            }
        }
    }

    // handling document class
    if docclass != nil {
        // add to the document
        errFn := documentBuilder.addDocumentClass(*docclass.(*Line))
        if errFn != nil {
            return nil, errFn(*startCoord, *endCoord)
        }
    }

    // handling preamble
    if preamble != nil {
        for _, line := range preamble.([]Line) {
            var component Component = &line
            errFn := documentBuilder.addPreambleLine(component)
            if errFn != nil {
                return nil, errFn(*startCoord, *endCoord)
            }
        }
    }

    // handling document content
    if content != nil {
        for i, component := range content.([]Component) {
            var errFn CreateError
```

```
            if i + 1 < len(content.([]Component)) {
                errFn = documentBuilder.addPreambleLine(component)
            } else {
                errFn = documentBuilder.addDocumentContent(component)
            }
            if errFn != nil {
                return nil, errFn(*startCoord, *endCoord)
            }
        }
    }

    // additional checks over the document occur during creation
    document, errFn := documentBuilder.buildDocument()
    if errFn != nil {
        return nil, errFn(*startCoord, *endCoord)
    }
    return document, nil
}

// Handles commands that precede the document class
// These should be rarely used in most LaTeX documents
PrerequisiteContent <- lines:MultiLaTeXLines {
    return lines, nil
}

// Handles the document class i.e. overall categorisation of the LaTeX document
DocumentClass <- command_start name:`documentclass` args:MultiArgument {
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }

    // parse data
    arguments := args.([]Argument)
    line := newLine(string(name.([]byte)), arguments, *startCoord, *endCoord)
    return line, nil
} / text {
    return nil, nil
}

// Handles metadata (i.e. the head) about the LaTeX document
Preamble <- lines:MultiSimpleLaTeXLines {
    return lines, nil
}

// Handles the overall content (i.e. the body) of the LaTeX document
DocumentContent <- lines:MultiLaTeXLines {
    return lines, nil
}

// Ensures the document does not match any further
// i.e. it ensures the End-of-File has been reached
EOF <- !.

/**
 * This section deals with LaTeX lines that require grouped contexts.
 * For example, all document content should be grouped under the 'document'.
 */
MultiLaTeXLines <- lines:LaTeXLine* {
    // creating coordinates for potential error handling
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }
```

```
    var result []Component
    stack := newStack[Component]()
    for _, line := range lines.([]any) {
        if line == nil {
            continue
        }

        // Handling ending groups
        coordinate, ok := line.(Coordinate)
        if ok {
            group, err := stack.Pop()
            if err != nil {
                return nil, err(*startCoord, *endCoord)
            }

            // updating group's end
            group.SetEndCoordinate(coordinate)
            continue
        }

        // adding component to pre-existing groups if necessary
        component, ok := line.(Component)
        if !ok || component == nil {
            return nil,
SERVER_RESPONSIBLE_NON_COMPONENT_DATA_STRUCTURE(*startCoord, *endCoord)
        }
        if !stack.IsEmpty() {
            peek, err := stack.Peek()
            if err != nil {
                return nil, err(*startCoord, *endCoord)
            }
            peer, ok := peek.(*Group)
            if !ok {
                return nil,
SERVER_RESPONSIBLE_STACK_CONTAINS_NON_GROUP_ELEMENT(*startCoord, *endCoord)
            }
            peer.AddComponent(component)
            component.SetGroup(peer)

        // add to the result if this is the topmost layer
        } else {
            result = append(result, component)
        }

        // handling grouped contexts
        _, ok = component.(*Group)
        if ok {
            stack.Push(component)
            continue
        }
    }
    return result, nil
}

LaTeXLine <- text line:(BeginGroup / EndGroup / SimpleLaTeXLine) text {
    return line, nil
}

BeginGroup <- command_start `begin` args:MultiArgument {
    // retrieving useful data
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
```

```
    }
    arguments := args.([]Argument)

    // retrieving name of the group
    var name string
    found := false
    for i, arg := range arguments {
        class, ok := arg.(*ClassArgument)
        if ok {
            name = strings.Trim(class.GetValue().(string), WHITESPACE)
            arguments[i] = nil
            found = true
            break
        }
    }
    if !found {
        return nil, MISSING_BEGIN_GROUP_NAME(*startCoord, *endCoord)
    }

    // handling groups ordering
    arguments = RemoveNilElements[Argument](arguments, nil)
    group := newGroup(name, arguments, *startCoord, *endCoord)
    groupStack.Push(group)
    return group, nil
}

EndGroup <- command_start `end` args:MultiArgument {
    // creating coordinates for potential error handling
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }
    arguments := args.([]Argument)

    // retrieving name of the group
    var name string
    found := false
    for _, arg := range arguments {
        class, ok := arg.(*ClassArgument)
        if ok {
            name = strings.Trim(class.GetValue().(string), WHITESPACE)
            found = true
            break
        }
    }
    if !found {
        return nil, MISSING_END_GROUP_NAME(*startCoord, *endCoord)
    }

    // checking a begin statement exists
    peek, err := groupStack.Peek()
    if err != nil {
        return nil, GROUP_BEGIN_MISSING(*startCoord, *endCoord)
    }

    // checking the group beginning matches its end
    if peek.GetName() != name {
        return nil, GROUP_NAME_MISMATCH(*startCoord, *endCoord)
    }
    groupStack.Pop()

    // requires a non-nil value to indicate it is not an error
    // we supply endCoord so we can update the group coordinates to account for
all the inner lines
```

```
    return *endCoord, nil
}

/**
 * This section handles simple LaTeX lines that do not require knowing about the
other lines to be processed.
 */
MultiSimpleLaTeXLines <- lines:SimpleWrappedLaTeXLine* {
    return AnyInterfaceToTSlice[Line](lines), nil
}

SimpleWrappedLaTeXLine <- text line:SimpleLaTeXLine text {
    return *line.(*Line), nil
}

// Handles any LaTeX line and provides it a data structure to be parsed further
// disallow 'documentclass', 'begin', and 'end' since they are handled
separately
SimpleLaTeXLine <- command_start !`documentclass` !`begin` !`end` name:word+
args:MultiArgument {
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }

    arguments := args.([]Argument)
    line := newLine(AnyInterfaceToString(name), arguments, *startCoord,
*endCoord)
    return line, nil
}

/**
 * This section deals with argument handling of individual LaTeX commands.
 * Namely, components within them that are surrounded by curly braces '{}' or
square brackets '[]'.
 * This section also includes handling for multiple arguments one after the
other.
 */

// Handles zero or more arguments in a row, directly one after the other
MultiArgument <- arguments:Argument* {
    return AnyInterfaceToTSlice[Argument](RemoveNilElements[any](arguments.
([]any), nil)), nil
}

// Handles either possible arguments, ensuring a uniform method of management
Argument <- argument:(OptionArgument / ClassArgument) {
    argumentStack = newStack[string]()
    return argument, nil
}

// For our purposes, a class argument is defined as one surrounded by curly
braces '{}'
ClassArgument <- '{' _ class:ArgumentContent _ '}' {
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }
    if !argumentStack.IsEmpty() {
        return nil, INNER_ARGUMENT_END_MISSING(*startCoord, *endCoord) // custom
err later
    }
    argument := newClassArgument(class.(string))
    return argument, nil
```

```
// Handling if the closing brace '}' is missing
} / '{' _ ArgumentContent {
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }
    return nil, CLASS_ARGUMENT_END_MISSING(*startCoord, *endCoord)
} / ArgumentContent _ '}' {
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }
    return nil, CLASS_ARGUMENT_START_MISSING(*startCoord, *endCoord)
}

// For our purposes, an option argument is defined as one surrounded by square
brackets '[]'
OptionArgument <- '[' _ options:ArgumentContent _ ']' {
    argument := newOptionArgument(options.(string))
    return argument, nil

// Handling if the closing bracket ']' is missing
} / '[' _ ArgumentContent {
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }
    return nil, OPTIONS_ARGUMENT_END_MISSING(*startCoord, *endCoord)
} / ArgumentContent _ ']' {
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }
    return nil, OPTIONS_ARGUMENT_START_MISSING(*startCoord, *endCoord)
}

/**
 * This section handles inner content of arguments. We do not need to parse the
data into their own structures, only match against required formatting.
 */

// Handling of the full inner content of an argument
ArgumentContent <- ArgumentPart* {
    return string(c.text), nil
}

// Handling of argument 'edges' such as {} or [], as well as allowing simple
LaTeX lines and ordinary text to pass through
ArgumentPart <- SimpleLaTeXLine / non_argument_content / ArgumentOpen /
ArgumentClose

// Handles opening argument edges, utilising a stack
ArgumentOpen <- str:argument_opener {
    chr := string(str.([]byte))
    argumentStack.Push(chr)
    return chr, nil
}

ArgumentClose <- &{
    // only match against the ending if there is a starting component
    // otherwise, we assume it is to be matched by the argument itself rather
than its inner contents
    if argumentStack.IsEmpty() {
```

```
        return false, nil
    }
    return true, nil
} str:argument_closer {
    startCoord, endCoord, err := GetCoordinates(c.pos.line, c.pos.col, c.text)
    if err != nil {
        return nil, DummyError(err)
    }

    argMapping := map[string]string{
        "[": "]",
        "{": "}",
    }

    closeChr := string(str.([]byte))
    openChr, errFn := argumentStack.Pop()
    if errFn != nil {
        return nil, errFn(*startCoord, *endCoord)
    }
    expectedCloseChr, ok := argMapping[openChr]
    if !ok {
        return nil, INNER_ARGUMENT_END_MISSING(*startCoord, *endCoord)
    }
    if expectedCloseChr != closeChr {
        return nil, INNER_ARGUMENT_MISMATCH(*startCoord, *endCoord)
    }
    return nil, nil
}

/**
 * This section deals with character classes which are used to indicate a group
of characters, or the lack thereof.
 * It serves a purpose of improving maintainability and readability of the
overall PEG script.
 */
// Handles argument edges
argument_opener <- [{[]
argument_closer <- [}\]]
non_argument_content <- [^\\{[\]}]

// Handles supported commands, currently only including \
command_start <- '\\'
non_special <- [^\\]
text "zero or more non-special characters" <- (command_start (& non_whitespace
non_word) / non_special)*

// Handles word characters
word <- [a-zA-Z0-9_]
non_word <- [^a-zA-Z0-9_]

// whitespace handling
non_whitespace <- [^ \n\t\r]
whitespace <- [ \n\t\r]
_ "zero or more whitespace characters" <- whitespace*
```

A.9 Agile sprint focuses

Agile sprints used during this project include:

- Creating the initial draft for the overview diagram for this project, with the purpose of showcasing, at a glance, the main objectives of LinTEX.
- Starting the ethics application. For this process, draft consent form (found in section A.4) and participant information sheet (PIS) (found in section A.3) were created.
- Creating the minimum viable product (MVP) to identify lack of alternative text usage, originally conforming to the 'accessibility' package, and implementing it through the necessary steps.
- Writing the November progress report.
- Improving the backend's parsing to be more maintainable through the use of design patterns and restructuring of file management.
- Implementing sophisticated error handling to ensure separate outputs for either specific, contextualised errors or descriptive, context-free error messages.
- Creating the final draft of the overview diagram.
- Completing the ethics application to gain ethics approval for the user study (section 5).
- Designing and implementing the graphical user interface (GUI) using Tailwind CSS and Flowbite.
- Conducting the user study to determine LinTEX's usability and its effectiveness in improving a LATEX document's accessibility for screen reader users.
- Implementing a few suggested fixes, depending on what the timeframe allowed, for usability concerns identified in the user study (section 5).
- Writing the final report, namely this one.

A.10 Functional requirements

Functional requirements include:

- LinTEX must allow one LATEX file to be uploaded.
- LinTEX must allow the file to be downloaded once the user has finished using the system.
- LinTEX should allow the finished document to be previewed.
- LinTEX must detect if tagging exists or not.
- LinTEX must recommend a fix by inserting tagging.
- LinTEX must detect when alt text is missing for graphics.
- LinTEX must recommend a fix by inserting alt text for graphics.
- LinTEX must detect when tagging is missing for tables.
- LinTEX must recommend tagging for tables.
- LinTEX should reference further reading for screen reader accessibility recommendations.
- The website should have all non-text content contain an alt text (WCAG 2.2: Guideline 1.1) W3C 2018.
- LinTEX may be usable without a keyboard.
- LinTEX should provide context-free, descriptive error messages.
- LinTEX should provide contextualised, simple error messages.

A.11 Non-functional requirements

Non-functional requirements include:

- The website should load within 2 seconds.
- The website should support a variety of standard desktop screen sizes.
- The website should support a variety of browsers.
- The website should be presentable in a simpler format if required (WCAG 2.2: Guideline 1.3) W3C 2018.
- The website should allow for content to be distinguishable including supporting resizing content and minimal colour usage (WCAG 2.2: Guideline 1.4) W3C 2018.
- The website may be operable through the keyboard alone (WCAG 2.2: Guideline 2.1) W3C 2018.
- The website should include navigation features (WCAG 2.2: Guideline 2.4) W3C 2018.
- The website should be predictable (WCAG 2.2: Guideline 3.2) W3C 2018.
- LinTeX should provide a PDF document in under a minute.
- LinTeX should have thorough documentation describing its usage and code.

A.12 Error list

```
// Acts as a shorthand list of error functions
// Restructuring the ordering at this point may change an error's ID,
// However, the code is set up such that it can support such a change
var (
      /**
       * Parsing related error functions.
       */
      // Argument parsing errors
      OPTIONS_ARGUMENT_START_MISSING           = CustomErrorWrapper("The opening square bracket '[' is missing from the argument.", "You should have an associated opening square bracket '[' for your closing square bracket ']'. \n\nExamples:\n- \\documentclassdraft] could have this specific error fixed by adding '[' between 'class' and 'draft', creating \\documentclass[draft] -- the final result may be \\documentclass[draft]{article}\n- \ \includegraphicsartifact] could have this specific error fixed by adding ']' between 'graphics' and 'artifact', creating \\includegraphics[artifact] -- the final result may be \\includegraphics[artifact]{image.png}", newParseError)
      OPTIONS_ARGUMENT_END_MISSING             = CustomErrorWrapper("The closing square bracket ']' is missing from the argument.", "You should have an associated closing square bracket ']' for your opening square bracket '['. \n\nExamples:\n- \\documentclass[draft could have this specific error fixed by adding ']' to the end, creating \\documentclass[draft] -- the final result may be \\documentclass[draft]{article}\n- \\includegraphics[alt={Some alt text} could have this specific error fixed by adding ']' to the end, creating \ \includegraphics[alt={Some alt text}] -- the final result may be \ \includegraphics[alt={Some alt text}]{image.png}", newParseError)
      CLASS_ARGUMENT_START_MISSING             = CustomErrorWrapper("The opening curly braces '{' is missing from the argument.", "You should have an associated opening curly brace '{' for your closing curly brace '}'. \n\nExamples:\n- \\documentclassarticle} could have this specific error fixed by adding '{' between 'class' and 'article', creating \\documentclass{article}\n- \ \includegraphicsimage.png} could have this specific error fixed by adding '{' between 'graphics' and 'image.png', creating \\includegraphics{image.png}", newParseError)
      CLASS_ARGUMENT_END_MISSING               = CustomErrorWrapper("The closing curly braces '}' is missing from the argument.", "You should have an associated closing curly brace '}' for your opening curly brace '{'. \n\nExamples:\n- \\documentclass{article could have this specific error fixed by adding '}' to the end, creating \\documentclass{article}\n- \ \includegraphics{image.png could have this specific error fixed by adding '}' to the end, creating \\includegraphics{image.png}", newParseError)
      INNER_ARGUMENT_END_MISSING               = CustomErrorWrapper("A closing argument, '}' or ']' is missing from the argument.", "You should have an associated ending curly brace '}' or square bracket ']' in your inner argument. The inner argument is the content within your arguments.\n\nExamples:\n- \ \includegraphics[alt={some alt text]{image.png} could be fixed by adding '}' directly after 'argument', creating \\includegraphics[alt={some alt text}] {image.png}\n- \\DocumentMetadata{tagging=on,tagging-setup={activate/all} could be fixed by adding '}' to the end, creating \ \DocumentMetadata{tagging=on,tagging-setup={activate/all}}", newParseError)
      INVALID_ARGUMENT_CONTENT_KEY_VALUE_FORMAT = CustomErrorWrapper("This argument is not in key-value pair form. A pair consists of a key and value separated by '=', with each pair separated by ','", "The argument should be in key-value form.\n\nA pair consists of 2 values separated by '=', where the right value could be surrounded by curly braces '{}' with content within it.\nAn argument in key-value form contains many pairs, each separated by commas ','. \n\nExamples:\n- \\DocumentMetadata{food} is not in key-value form, whereas \ \DocumentMetadata{tagging=on} is.\n- \\includegraphics[good]{image.png} does not have its optional parameter, surrounded by square brackets '[]', in key-value form. However, \\includegraphics[alt={Some alt text}]{image.png} does have its optional parameter in key-value form.", newParseError)
      KEY_EMPTY                                = CustomErrorWrapper("The key provided is empty.", "An empty key has been provided to the argument. This can be fixed by simply adding a key to the relevant pair.\nExamples:\n- \
```

```
\DocumentMetadata{=on} is missing the key 'tagging', where the fixed version is
\\DocumentMetadata{tagging=on}\n- \\includegraphics[={Some alt text}]{image.png}
is missing the key 'alt', where \\includegraphics[alt={Some alt text}]}
{image.png} is the fixed version", newParseError)
      VALUE_EMPTY                                = CustomErrorWrapper("The value
provided is empty.", "An empty value has been provided to the argument. This can
be fixed by simply adding a value to the relevant pair.\nExamples:\n- \
\DocumentMetadata{tagging=} is missing the value 'on' or 'off', where the fixed
version can be \\DocumentMetadata{tagging=on}\n- \\includegraphics[alt=]
{image.png} is missing the value for the key 'alt', where \
\includegraphics[alt={Some alt text}]}{image.png} is a valid fixed version",
newParseError)
      INNER_ARGUMENT_MISMATCH                    = CustomErrorWrapper("There was
a mismatch from the starting and ending braces.", "There was a mismatch from the
starting and ending braces for the inner argument. The inner argument is the
content within an argument. This means two incompatible grouping methods have
been using, such as an opening square bracket '[' together with a closing curly
brace '}'.\n\nExamples:\n-\\includegraphics[alt={Some alt text]]{image.png}
could be fixed by using \\includegraphics[alt={Some alt text}]{image.png}\n- \
\includegraphics[alt=[Some alt text}]{image.png} could be fixed by using \
\includegraphics[alt={Some alt text}]{image.png}", newParseError)

      // Group parsing errors
      GROUP_NAME_MISMATCH = CustomErrorWrapper("The class argument within \
\begin line did not match the class argument within \\end line.", "The ending
group name did not match the starting group name. This can be fixed by either
using the starting group name within the ending group, or vice versa.\nExamples:
\n- \\begin{tabular}...\\end{itemize} can be fixed by replacing 'itemize' with
'tabular', creating \\begin{tabular}...end{tabular}.\n- \\begin{tabular}...\
\end{itemize} can be fixed by replacing 'tabular' with 'itemize', creating \
\begin{itemize}...end{itemize}.", newParseError)
      GROUP_BEGIN_MISSING = CustomErrorWrapper("The matching \\begin line could
not be found for the \\end line.", "There is no beginning for the group. This
can be fixed by adding a beginning for the group.\n\nExample:\n- ...\
\end{tabular} by itself by can be fixed by adding a \\begin{tabular} to the
front, creating \\begin{tabular}...\\end{tabular}", newParseError)
      GROUP_END_MISSING   = CustomErrorWrapper("The matching \\end line could
not be found for the \\begin line.", "There is no ending for the group. This can
be fixed by adding an ending for the group.\n\nExample:\n- \\begin{tabular}...
by itself can be fixed by adding a \\end{tabular} to the end, creating \
\begin{tabular}...\\end{tabular}", newParseError)

      // Server-responsible parsing errors
      SERVER_RESPONSIBLE_STACK_EMPTY                          =
CustomErrorWrapper("Something went wrong on the server's end. The grouping stack
is empty. Please report this to the server owner.", "Something went wrong on the
server's end. The grouping stack is empty. Please report this to the server
owner.", newParseError)
      SERVER_RESPONSIBLE_STACK_CONTAINS_NON_GROUP_ELEMENT =
CustomErrorWrapper("Something went wrong on the server's end. The grouping stack
contained a non-group element. Please report this to the server owner.",
"Something went wrong on the server's end. The grouping stack contained a non-
group element. Please report this to the server owner.", newParseError)
      SERVER_RESPONSIBLE_NON_COMPONENT_DATA_STRUCTURE     =
CustomErrorWrapper("Something went wrong on the server's end. The data structure
for a LaTeX line did not return one that is supported. Please report this to the
server owner.", "Something went wrong on the server's end. The data structure
for a LaTeX line did not return one that is supported. Please report this to the
server owner.", newParseError)
      SERVER_RESPONSIBLE_TABLE_NOT_FOUND                      =
CustomErrorWrapper("Something went wrong on the server's end. The tabular group
could not be found. Please report this to the server owner.", "Something went
wrong on the server's end. The tabular group could not be found. Please report
this to the server owner.", newParseError)
```

```
      SERVER_RESPONSIBLE_INVALID_DOCUMENT_CLASS           =
CustomErrorWrapper("Something went wrong on the server's end. A non-document
class argument was provided as one. Please report this to the server owner.",
"Something went wrong on the server's end. A non-document class argument was
provided as one. Please report this to the server owner.", newParseError)

      // conversion parsing errors
      CANNOT_CONVERT_TO_PDF = CustomErrorWrapper("The file was not converted
into .pdf format. Please compile it on a TeX engine to identify the cause.", "An
error that is not handled on LinTeX's end had occurred during the compilation of
the document. You should use a dedicated environment to debug this error, such
as TeX Live or Overleaf.", newParseError)

      /**
       * Structure related error functions
       */
      // Missing essential element structure errors
      DOCUMENT_CLASS_NOT_FOUND                = CustomErrorWrapper("Missing \
\documentclass line.", "There was no \\documentclass line detected in the
document. This is necessary for the document to compile.\n\nExamples:\n- \
\documentclass[draft]{article} is a valid example\n- \\documentclass{book} is a
valid example", newStructureError)
      SEVERAL_DOCUMENT_CLASSES_FOUND          = CustomErrorWrapper("You cannot
have more than 1 document class within a single document.", "There should only
ever be 1 \\documentclass command within a single document. This error can be
alleviated by removing all document class commands that are not essential to the
document.\n\nExample:\n- \\documentclass{article}\\documentclass{book} could be
fixed by removing the 'book' document class, leaving only \
\documentclass{article}", newStructureError)
      DOCUMENT_CLASS_REQUIRED_ARGUMENT_MISSING = CustomErrorWrapper("\
\documentclass should have at least 1 required argument. For example, '\
\documentclass{article}' is a valid example.", "The document class currently
contains no required arguments indicating its type. This can be alleviated by
providing it with a valid typing.\n\nExample:\n- \\documentclass could be fixed
by adding the 'article' or 'book' types, creating \\documentclass{article} and \
\documentclass{book} respectively.", newStructureError)
      DOCUMENT_CONTENT_GROUP_NOT_FOUND        = CustomErrorWrapper("Missing \
\begin{document} and \\end{document} lines.", "The body of the document should
be contained within a 'document' group. Content outwith this group is considered
to be within the preamble.\n\nExample:\n- \\includegraphics[alt={Some alt text}]
{image.png} cannot exist outside the 'document' group, so \\begin{document} ...
\\includegraphics[alt={Some alt text}]{image.png} ... \\end{document} is a valid
solution if it exists outside the relevant group.", newStructureError)
      DOCUMENT_CONTENT_GROUP_NOT_DOCUMENT_TYPE = CustomErrorWrapper("Incorrect
group name used for the document content.", "It was expected to receive a
'document' group indicating the body of the document, but another grouping type
was provided instead. This could be caused by forgetting to include the
'document' group or having some typo exist somewhere.\n\nExample:\n- \
\begin{tabular} ... \\end{tabular} could require being encapsulated in a
'document' group, creating \\begin{document} ... \\begin{tabular} ... \
\end{tabular} ... \\end{document}", newStructureError)

      // Missing required arguments structure errors
      MISSING_BEGIN_GROUP_NAME = CustomErrorWrapper("You must have at least one
required argument '{}' with \\begin lines.", "A group beginning has no name
provided to it. This could be caused by a lack of required arguments, surrounded
by '{}'.\n\nExample:\n- \\begin by itself is invalid, whereas \\begin{tabular}
is valid as it was assigned the 'tabular' group name.", newStructureError)
      MISSING_END_GROUP_NAME   = CustomErrorWrapper("You must have at least one
required argument '{}' with \\end lines.", "A group ending has no name provided
to it. This could be caused by a lack of required arguments, surrounded by '{}'.
\n\nExample:\n- \\end by itself is invalid, whereas \\end{tabular} is valid as
it was assigned the 'tabular' group name.", newStructureError)
```

```
      // Containing group constructs within non-grouping contexts
      PREAMBLE_CONTAINS_GROUP     = CustomErrorWrapper("The preamble cannot
contain any grouping constructs such as \\begin or \\end.", "The preamble of the
document, the document outwith the 'document' group, cannot contain grouping
constructs in its outermost layer. This was likely caused by misplacing a
grouping construct that should be within the 'document' group.\nExample:\n- \
\usepackage{...} ... \\begin{tabular} ... \\end{tabular} ... \
\begin{document} ... \\end{document} could be fixed by moving the 'tabular'
group within the 'document' group, creating \\usepackage{...} ... \
\begin{document} ... \\begin{tabular} ... \\end{tabular} ... \\end{document}",
newStructureError)
      PREREQUISITE_CONTAINS_GROUP = CustomErrorWrapper("The content before the
document class cannot contain any grouping constructs such as \\begin or \
\end.", "The prerequisite content of the document, the document before the
documentclass, cannot contain grouping constructs. This was likely caused by
misplacing a grouping construct that should be within the 'document' group.
\nExample:\n- \\begin{tabular} ... \\end{tabular} ... \\documentclass{...} ... \
\begin{document} ... \\end{document} could be fixed by moving the 'tabular'
group within the 'document' group, creating ... \\documentclass{...} ... \
\begin{document} ... \\begin{tabular} ... \\end{tabular} ... \\end{document}",
newStructureError)

      // Extra latex code where it shouldn't belong
      DOCUMENT_CONTENT_ALREADY_EXISTS = CustomErrorWrapper("You cannot have more
than one document content component.", "You can only have one 'document' group
within the document. This error was caused by 2 or more 'document' groups
existing within the document.\n\nExample:\n- \\begin{document} ... \
\end{document} ... \\begin{document} ... \\end{document} which can be fixed by
either combining the groups, or removing one of them, both ending in the form \
\begin{document} ... \\end{document}", newStructureError)
      USEPACKAGE_OUTSIDE_PREAMBLE     = CustomErrorWrapper("\\usepackage should
only exist in the preamble.", "\\usepackage should only exist in the preamble.
\n\nExample:\n- \\documentclass{...} \\begin{document} ... \\usepackage{...} ...
\\end{document} can be fixed by moving the package to the preamble: \
\documentclass{...} ... \\usepackage{...} ... \\begin{document} ... \
\end{document}", newStructureError)
      USEPACKAGE_LACKS_CLASS_ARGUMENT = CustomErrorWrapper("\\usepackage should
have a required argument. For example, '\\usepackage{graphicx}' is a valid
example.", "\\usepackage should have a required argument. A required argument is
an argument surrounded by curly braces '{}'.\n\nExample:\n- \\usepackage by
itself can cause the error, with \\usepackage{graphicx} being a valid package,
as 'graphicx' is a valid package name.", newStructureError)

      /**
       * Accessibility related error functions
       */
      // tagging accessibility errors
      DOCUMENT_METADATA_MISSING               = CustomErrorWrapper("Missing \
\DocumentMetadata{tagging=on} line before \\documentclass.", "The document
metadata is not provided. This is necessary to ensure appropriate tagging of the
document takes place. A recommended document metadata command is as follows:
\n\n\\DocumentMetadata{\n\ttagging = on,\n\ttagging-setup = {activate/all},
\n\tlang=en-GB,\n\tpdfstandard = ua-2,\n\tpdfversion = 2.0\n}",
newAccessibilityError)
      DOCUMENT_METADATA_APPEARED_LATE         = CustomErrorWrapper("\
\DocumentMetadata should not appear after \\documentclass.", "The document
metadata must only exist before the document class.\n\nExample:\n- \
\documentclass{...} ... \\DocumentMetadata{...} is invalid due to its ordering,
yet \\DocumentMetadata{...} ... \\documentclass{...} is valid",
newAccessibilityError)
      DOCUMENT_METADATA_REPEATED              = CustomErrorWrapper("There should
only be one \\DocumentMetadata in a single LaTeX file.", "You should only
contain 1 document metadata in your document.\n\nExample:\n- \
\DocumentMetadata{...} ... \\DocumentMetadata{...} is invalid as there is more
```

```
than 1 document metadata, yet we can remove or combine them to create an
effective \\DocumentMetadata{...}", newAccessibilityError)
      METADATA_SEVERAL_ARGUMENTS             = CustomErrorWrapper("\
\DocumentMetadata should only have one required argument. For example, '\
\DocumentMetadata{tagging=on}'.", "The document metadata cannot contain more
than 1 argument, and its only argument should be a required one (surrounded by
curly braces '{}'). A recommended document metadata command is as follows:\n\n\
\DocumentMetadata{\n\ttagging = on,\n\ttagging-setup = {activate/all},
\n\tlang=en-GB,\n\tpdfstandard = ua-2,\n\tpdfversion = 2.0\n}",
newAccessibilityError)
      METADATA_OTHER_ARGUMENTS_UNSUPPORTED    = CustomErrorWrapper("\
\DocumentMetadata should not have other argument types, such as those surrounded
by square brackets '[]'.", "The document metadata should not contain other
arguments types, such as optional arguments surrounded by square brackets. It
should contain only one argument which is a required one (surrounded by curly
braces '{}'). A recommended document metadata command is as follows:\n\n\
\DocumentMetadata{\n\ttagging = on,\n\ttagging-setup = {activate/all},
\n\tlang=en-GB,\n\tpdfstandard = ua-2,\n\tpdfversion = 2.0\n}",
newAccessibilityError)
      METADATA_LACKS_CLASS_ARGUMENT           = CustomErrorWrapper("\
\DocumentMetadata should have one required argument.", "The document metadata
lacks a required argument. There should be exactly one required argument
(surrounded by curly braces '{}') passed to \\DocumentMetadata. A recommended
document metadata command is as follows:\n\n\\DocumentMetadata{\n\ttagging = on,
\n\ttagging-setup = {activate/all},\n\tlang=en-GB,\n\tpdfstandard =
ua-2,\n\tpdfversion = 2.0\n}", newAccessibilityError)
      METADATA_LACKS_ENABLED_TAGGING_ARGUMENT = CustomErrorWrapper("\
\DocumentMetadata should have 'tagging=on' within its required argument.", "The
document metadata should contain the pair with key 'tagging' and value 'on'. A
recommended document metadata command is as follows:\n\n\
\DocumentMetadata{\n\ttagging = on,\n\ttagging-setup = {activate/all},
\n\tlang=en-GB,\n\tpdfstandard = ua-2,\n\tpdfversion = 2.0\n}",
newAccessibilityError)
      METADATA_LACKS_TAGGING_SETUP_ARGUMENT   = CustomErrorWrapper("\
\DocumentMetadata should have 'tagging-setup={...}' within its required
argument. It is recommended to use 'tagging-setup={activate/all}'.", "The
document metadata should contain the pair with key 'tagging-setup' and a value,
surrounded by curly braces '{}', obtained from
https://ctan.math.washington.edu/tex-archive/macros/latex/contrib/tagpdf/
tagpdf.pdf. A recommended document metadata command is as follows:\n\n\
\DocumentMetadata{\n\ttagging = on,\n\ttagging-setup = {activate/all},
\n\tlang=en-GB,\n\tpdfstandard = ua-2,\n\tpdfversion = 2.0\n}",
newAccessibilityError)
      METADATA_LACKS_PDF_STANDARD_ARGUMENT    = CustomErrorWrapper("\
\DocumentMetadata should have 'pdfstandard=...' within its required argument. It
is recommended to use 'pdfstandard=ua-2'.", "A PDF standard should be specified.
This utilises the key 'pdfstandard' and a value based on available standards. A
recommended document metadata command is as follows:\n\n\
\DocumentMetadata{\n\ttagging = on,\n\ttagging-setup = {activate/all},
\n\tlang=en-GB,\n\tpdfstandard = ua-2,\n\tpdfversion = 2.0\n}",
newAccessibilityError)
      METADATA_LACKS_LANGUAGE_ARGUMENT        = CustomErrorWrapper("\
\DocumentMetadata should have 'lang=...' within its required argument. For
example, 'lang=en-GB' is a valid option.", "The language of the document should
be provided using the key 'lang' and an appropriate value from
https://developer.mozilla.org/en-US/docs/Glossary/BCP_47_language_tag. A
recommended document metadata command is as follows:\n\n\
\DocumentMetadata{\n\ttagging = on,\n\ttagging-setup = {activate/all},
\n\tlang=en-GB,\n\tpdfstandard = ua-2,\n\tpdfversion = 2.0\n}",
newAccessibilityError)
      METADATA_LACKS_PDF_VERSION              = CustomErrorWrapper("\
\DocumentMetadata should have 'pdfversion=...' within its required argument. It
is recommended to use 'pdfversion=2.0' for compatibility with
pdfstandard=ua-2.", "A suitable PDF version should be provided, through the key
```

```
'pdfversion' and value chosen based on the PDF standard the document utilises. A
recommended document metadata command is as follows:\n\n\
\DocumentMetadata{\n\ttagging = on,\n\ttagging-setup = {activate/all},
\n\tlang=en-GB,\n\tpdfstandard = ua-2,\n\tpdfversion = 2.0\n}",
newAccessibilityError)

      // image-related alt text accessibility errors
      GRAPHICS_MISSING_ARGUMENTS            = CustomErrorWrapper("\
\includegraphics should at least 1 argument.", "\\includegraphics requires at
least one argument for the file location of the graphic.\n\nExample:\n- \
\includegraphics is invalid by itself, whereas \\includegraphics{image.png} is a
valid example", newAccessibilityError)
      GRAPHICS_TOO_MANY_ARGUMENTS           = CustomErrorWrapper("\
\includegraphics should have at most 2 arguments.", "\\includegraphics currently
has more than 2 arguments, when the most arguments that can be supported is 2
(with an optional and required argument each).\n\nExample:\n- \
\includegraphics[alt={Some alt text}]{image.png} is an example of using 2
arguments to specify 'Some alt text' alt text, and 'image.png' file location",
newAccessibilityError)
      GRAPHICS_FIRST_ARGUMENT_NOT_OPTIONAL  = CustomErrorWrapper("\
\includegraphics should have its first of both arguments being optional. For
example, '\\includegraphics[...]{...}' is a valid format.", "The first argument
of \\includegraphics, in the context of having 2 arguments, should be an
optional argument (surrounded by square brackets '[]').\n\nExample:\n- \
\includegraphics{alt={Some alt text}}{image.png} is invalid as its first
argument should be optional, creating \\includegraphics[alt={Some alt text}]
{image.png}", newAccessibilityError)
      GRAPHICS_SECOND_ARGUMENT_NOT_REQUIRED = CustomErrorWrapper("\
\includegraphics should have its second of both arguments being required. For
example, '\\includegraphics[...]{...}' is a valid format.", "The second argument
of \\includegraphics, in the context of having 2 arguments, should be a required
argument (surrounded by curly braces '{}').\n\nExample:\n- \
\includegraphics[alt={Some alt text}][image.png] is invalid as its second
argument should be required, creating \\includegraphics[alt={Some alt text}]
{image.png}", newAccessibilityError)
      GRAPHICS_OUTSIDE_DOCUMENT_CONTENT     = CustomErrorWrapper("\
\includegraphics should only appear within the document content.", "\
\includegraphics cannot be contained outwith the document content, i.e. the
'document' group.\n\nExample:\n- \\includegraphics[...]{...} ... \
\begin{document} ... \\end{document} is invalid, whereas \\begin{document} ... \
\includegraphics[...]{...} ... \\end{document} is valid", newAccessibilityError)
      GRAPHICS_LACKS_ALT_TEXT               = CustomErrorWrapper("\
\includegraphics should contain an optional argument containing alternative text
('alt text'), actual text, or artifact. For example, '\\includegraphics[alt={An
image of an apple}]{apple.png} is an accessible graphic.", "\\includegraphics
should contain an optional argument containing alternative text ('alt text'),
actual text, or artifact.\n\nAlternative text is text to be used by screen
readers, or if the relevant graphic does not load.\nActual text is text that
replaces the image from a screen reader's perspective. It is read as ordinary
text.\nArtifact is used to ignore the image when read by a screen reader. This
is used in the context of decorative images.\n\nSome examples include:\n- \
\includegraphics[alt={An image of an apple}]{apple.png}\n- \
\includegraphics[actualtext={Orange}]{orange.png}\n- \\includegraphics[artifact]
{luigi.png}", newAccessibilityError)
      GRAPHICS_LACKS_SOURCE                 = CustomErrorWrapper("\
\includegraphics should contain a required argument for its source. For example,
'\\includegraphics[alt={Some alt text}]{image.png}' is a valid graphic.", "\
\includegraphics contains exactly one optional argument, when an additional
argument for the source was expected.\n\nExample:\n- \\includegraphics[alt={Some
alt text}] is invalid, but adding a source to create \\includegraphics[alt={Some
alt text}]{image.png} makes it valid", newAccessibilityError)
      MISSING_GRAPHICX_PACKAGE              = CustomErrorWrapper("\
\includegraphics requires the 'graphicx' package. This can be included by
inserting '\\usepackage{graphicx}' into the preamble.", "The 'graphicx' package
```

```
is required to use \\includegraphics. This should be included in the preamble.
\n\nExample:\n- \\documentclass{...} ... \\usepackage{graphicx} ... \
\begin{document} ... \\includegraphics[alt={Some alt text}]{image.png} ... \
\end{document}", newAccessibilityError)

      // table-related accessibility errors
      TABLE_LACKS_REQUIRED_ARGUMENT                    =
CustomErrorWrapper("Tabular groups should have an additional required
argument.", "Tabular groups should contain an additional required argument
indicating the structure of its columns. This occurs immediately after \
\begin{tabular}.\n\nExample:\n\\begin{tabular}{ccc}\n\tHeading 1 & Heading 2 &
Heading 3 \\\\\\hline\n\t1 & 2 & 3 \\\\\n\t4 & 5 & 6\\\\\n\\end{tabular}",
newAccessibilityError)
      TABLE_CANNOT_PARSE_COLUMNS                       =
CustomErrorWrapper("Could not parse how many columns the table has.", "unused.",
newAccessibilityError)
      TABLE_MISSING_TAG_PDF_SETUP                      =
CustomErrorWrapper("Tabular groups should have \\tagpdfsetup{...} the line
before the group. It should contain 'table/tagging=presentation' or
'table/header-rows={N}' where N is the row the heading occurs.", "Tabular groups
should have \\tagpdfsetup{...} the line before the group. It should contain
'table/tagging=presentation' or 'table/header-rows={N}' where N is the row the
heading occurs. 'table/tagging=presentation' should be used when you do not want
the screen reader to read the relevant heading before each table cell, whereas
'table/header-rows={N}' should be used if you do want that effect. N should be
1-indexed, such that the first row is N = 1.\n\nExample 1:\n\
\tagpdfsetup{table/header-rows={1}}\n\\begin{tabular}{ccc}\n\tHeading 1 &
Heading 2 & Heading 3 \\\\\\hline\n\t1 & 2 & 3 \\\\\n\t4 & 5 & 6\\\\\n\
\end{tabular}\n\nExample 2:\\tagpdfsetup{table/tagging=presentation}\n\
\begin{tabular}{ccc}\n\tHeading 1 & 1 & 2 \\\\\n\tHeading 2 & 3 & 4\\\\\n\
\end{tabular}", newAccessibilityError)
      TAG_PDF_SETUP_REQUIRED_ARGUMENT                  = CustomErrorWrapper("\
\tagpdfsetup should have exactly one required argument.", "\\tagpdfsetup should
have exactly one required argument. A required argument is an argument
surrounded by curly braces '{}'. It should contain 'table/tagging=presentation'
or 'table/header-rows={N}' where N is the row the heading occurs.
'table/tagging=presentation' should be used when you do not want the screen
reader to read the relevant heading before each table cell, whereas
'table/header-rows={N}' should be used if you do want that effect. N should be
1-indexed, such that the first row is N = 1.\n\nExamples:\n- \
\tagpdfsetup{table/tagging=presentation}\n- \\tagpdfsetup{table/header-rows={1}}
when the table's headings are in the first row.", newAccessibilityError)
      TAG_PDF_SETUP_NON_REQUIRED_ARGUMENT              = CustomErrorWrapper("\
\tagpdfsetup should an argument surrounded by curly braces '{}'.", "\
\tagpdfsetup cannot have optional arguments arguments, instead it should have
exactly one required argument. A required argument is an argument surrounded by
curly braces '{}'. It should contain 'table/tagging=presentation' or
'table/header-rows={N}' where N is the row the heading occurs.
'table/tagging=presentation' should be used when you do not want the screen
reader to read the relevant heading before each table cell, whereas
'table/header-rows={N}' should be used if you do want that effect. N should be
1-indexed, such that the first row is N = 1.\n\nExamples:\n- \
\tagpdfsetup{table/tagging=presentation}\n- \\tagpdfsetup{table/header-rows={1}}
when the table's headings are in the first row.", newAccessibilityError)
      TAG_PDF_SETUP_HEADER_ROWS_INVALID_VALUE          = CustomErrorWrapper("\
\tagpdfsetup{table/header-rows=...} should have an integer value surrounded by
curly braces as the inner argument. For example, \\tagpdfsetup{table/header-
rows={1}} indicates the first row is the heading row.",
"\\tagpdfsetup{table/header-rows=...} should have a value of form '{N}' where N
is the row the headings occur.\n\nExample:\n- \\tagpdfsetup{table/header-
rows={1}} where the headings occur in the first row", newAccessibilityError)
      TAG_PDF_SETUP_LACKS_HEADER_ROWS_OR_PRESENTATION = CustomErrorWrapper("\
\tagpdfsetup should have an argument with 'table/header-rows={N}' with N being
the row the heading occurs, or 'table/tagging=presentation' to indicate the
```

```
screen reader is not to read the table.", "\\tagpdfsetup lacks an appropriate
setting within its required argument. It should contain
'table/tagging=presentation' or 'table/header-rows={N}' where N is the row the
heading occurs. 'table/tagging=presentation' should be used when you do not want
the screen reader to read the relevant heading before each table cell, whereas
'table/header-rows={N}' should be used if you do want that effect. N should be
1-indexed, such that the first row is N = 1.\n\nExamples:\n- \
\tagpdfsetup{table/tagging=presentation}\n- \\tagpdfsetup{table/header-rows={1}}
when the table's headings are in the first row.", newAccessibilityError)
      TAG_PDF_SETUP_TABLE_TAGGING_INVALID_VALUE       = CustomErrorWrapper("\
\tagpdfsetup{table/tagging=...} should have the inner value 'presentation'
creating '\\tagpdfsetup{table/tagging=presentation}'.",
"\\tagpdfsetup{table/tagging=...} contains an invalid value. The expected value
was 'presentation' such that \\tagpdfsetup{table/tagging=presentation} would be
the result.", newAccessibilityError)

      /**
       * Server-created errors. These can be caused by server-related processes
failing
       */
      SERVER_RESPONSIBLE_CANNOT_FIND_WORKING_DIRECTORY =
CustomErrorWrapper("Something went wrong on the server's end. The working
directory could not be retrieved. Please report this to the server owner.",
"Something went wrong on the server's end. The working directory could not be
retrieved. Please report this to the server owner.", newServerError)
      SERVER_RESPONSIBLE_WRITE_FILE_ERROR             =
CustomErrorWrapper("Something went wrong on the server's end. Could not write to
the file. Please report this to the server owner.", "Something went wrong on the
server's end. Could not write to the file. Please report this to the server
owner.", newServerError)
      SERVER_RESPONSIBLE_READ_FILE_ERROR              =
CustomErrorWrapper("Something went wrong on the server's end. The file location
for the LaTeX file could not be read. Please report this to the server owner.",
"Something went wrong on the server's end. The file location for the LaTeX file
could not be read. Please report this to the server owner.", newServerError)
      SERVER_RESPONSIBLE_UNIDENTIFIED_ERROR           =
CustomErrorWrapper("Something went wrong on the server's end. There is an
unidentified error. Please report this to the server owner.", "Something went
wrong on the server's end. There is an unidentified error. Please report this to
the server owner.", newServerError)
      SERVER_RESPONSIBLE_INVALID_STRING_ERROR         =
CustomErrorWrapper("Something went wrong on the server's end. Could not parse
string. Please report this to the server owner.", "Something went wrong on the
server's end. Could not parse string. Please report this to the server owner.",
newServerError)
      SERVER_RESPONSIBLE_INVALID_REGEX_ERROR          =
CustomErrorWrapper("Something went wrong on the server's end. Could not parse
regular expression. Please report this to the server owner.", "Something went
wrong on the server's end. Could not parse regular expression. Please report
this to the server owner.", newServerError)
)
```